

Python for Data Science

Course Reference Guide & Notes



EDITION 4.2



Variables

December 30, 2022

1 Variables

Video Tutorials:

[Function ID in Python](#)

In Python variables work more like tags. When you do an assignment in Python, it tags the value with the variable name.

```
[1]: a = 1
```



When Python executes `a = 1`, it creates int object and creates an object reference called `a`, we can say that variable `a` has been assigned int 1. The `=` operator binds an object in a memory.

```
[2]: a = 2
```

If you change the value of the variable, it just changes the tag to the new value in memory.



When a value is without names/tags it is automatically removed from memory by Python's Automatic Garbage Collection.

```
[3]: b = a
```

Assigning one variable to another makes a new tag bound to the same value. If object reference already exists, it is simply re-bound to refer to the object on the right of the `=` operator.



If object reference does not exist, it is created by the = operator

A value will have only one copy in memory and all the variables having this value will refer to this memory location.

For example when you have variables a, b, c having a value 10, it doesn't mean that there will be 3 copy of 10s in memory. There will be only one 10 and all the variables a, b, c will point to this value.

Function id() will return an object memory address or object's identity.

```
[28]: a = 10  
      id(a)
```

```
[28]: 4503726720
```

```
[29]: b = 10  
      id(b)
```

```
[29]: 4503726720
```

```
[30]: c = 10  
      id(c)
```

```
[30]: 4503726720
```

Once a variable is updated, say you are doing $a = a + 1$ a new value 11 will be allocated in memory and the variable a will be pointing to 11.

```
[31]: a = a + 1  
      id(a)
```

```
[31]: 4503726752
```

```
[32]: a
```

```
[32]: 11
```

Python implementation keeps an array of integer objects for all integers between -5 and 256. So when we create an integer in that range, they simply back reference to the existing object.

2 Let's take a look at strings now.

```
[33]: s1 = 'apple'
      s2 = 'apple'
      id(s1), id(s2)
```

```
[33]: (140543040089456, 140543040089456)
```

When the strings are simple and short variable names are referring to the same object in memory.

```
[34]: s1 == s2
```

```
[34]: True
```

```
[35]: s3 = 'hello, world!'
      s4 = 'hello, world!'
      s3 == s4
```

```
[35]: True
```

```
[36]: id(s3), id(s4)
```

```
[36]: (140543041774832, 140543041775280)
```

This is called interning, and Python does interning (to some extent) of shorter string literals (as in s1 and s2) which are created at compile time. But in general, Python string literals creates a new string object each time (as in s3 and s4). Interning is runtime dependant and is always a trade-off between memory use and the cost of checking if you are creating the same string.

3 The type() function returns the data type of the data item

```
[27]: type(total)
```

```
[27]: str
```

Python is dynamically typed language. “Typing” refers to “type” as in data type. Dynamic typed languages DO NOT require the explicit declaration of the variables before they used.

In Python type checking is done at run time vs static languages - done at complete time.

In Python you can assign any kind of data to any variable.

Python Strings

December 30, 2022

1 Python Strings

Video Tutorials:

[Python String](#)

[is_alpha\(\) String Method](#)

[Filtering a String with Python](#)

Strings in python are surrounded by either single quotation marks, or double quotation marks.

```
[1]: 'hello'
```

```
[1]: 'hello'
```

is the same as

```
[2]: "hello"
```

```
[2]: 'hello'
```

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

```
[3]: word = "apple"  
     print(word)
```

apple

```
[4]: type(word)
```

```
[4]: str
```

Let me run a couple of examples to illustrate Integers, Floats and Strings. In a cell try adding “7” to an integer,

7 + “7” will give us an error message.

```
[5]: 7 + "7"
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-5-62feab9aa80e> in <module>  
----> 1 7 + "7"  
  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Python is telling us that we cannot mix different data types. It is impossible to add a string to an integer. Literally it's telling us not to use a plus sign between integers and strings. There are a couple of things we can do to fix this. We can try to convert one data type into another. Since "7" looks like a number we can use the built-in function `int()` to convert it to a numeric data type. If we had a letter or some other character instead of "7" that would not be possible.

```
[6]: 7 + int(7)
```

```
[6]: 14
```

We can see that string "7" is being converted to an integer. Integer 7 plus integer 7 expectedly gets us 14. If we had a letter or some other character instead of "7" the conversion to a numeric type would not be possible.

Alternative option would be to convert "7" to a float. Using function `float()`

```
[7]: 7 + float("7")
```

```
[7]: 14.0
```

This time we got 14.0 as a result of integer 7 is being added to a float. At the end of the day string "7" was converted to a float 7.0. In arithmetical operation like this, $7 + 7.0$, float always wins, and the result would be returned as a float.

There is one more option we can do here. We can convert integer to a string with the help of the built-in function `str()`.

```
[8]: str(7) + "7"
```

```
[8]: '77'
```

You can see that now we have joined or concatenated one string to another. Integer 7 was converted to a string and the whole operation yielded a new string "77".

The main idea behind these three examples is that the same plus operator brings about different results based on the data type it is applied to. All data types behave differently. Another words each data type has its own features. This totally makes sense because we understand that a plastic bottle and a cup are two different containers that behave in their own certain ways.

Python Numeric Data Types

December 30, 2022

1 Python Build-in Data Types

2 Numeric types

Video Tutorials:

[Numeric Data Types in Python](#)

[Division in Python](#)

In computer science and computer programming, a data type or simply type is an attribute of data which tells Python how the programmer intends to use the data.

There are three numeric types in Python:

```
int
float
complex
```

3 Int or Integer

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

```
[2]: x = 7
      y = 1000000
      z = -10

      print(type(x))
      print(type(y))
      print(type(z))
```

```
<class 'int'>
<class 'int'>
<class 'int'>
```

4 Float

Float, or “floating point number” is a number, positive or negative, containing one or more decimals.

```
[3]: x = 2.5
      y = 100.99
      z = -35.59

      print(type(x))
      print(type(y))
      print(type(z))
```

```
<class 'float'>
<class 'float'>
<class 'float'>
```

5 Complex

An complex number is represented by “ $x + yi$ “. Python converts the real numbers x and y into complex using the function `complex(x,y)`. The real part can be accessed using the function `real()` and imaginary part can be represented by `imag()`.

```
[4]: x = 3+5j
      y = 5j
      z = -5j

      print(type(x))
      print(type(y))
      print(type(z))
```

```
<class 'complex'>
<class 'complex'>
<class 'complex'>
```

Complex numbers which are mostly used where we are using two real numbers. For instance, an electric circuit which is defined by voltage(V) and current(C) are used in geometry, scientific calculations and calculus.

6 Python Arithmetic Operators

Operator	Name	Example
+	Addition	$2 + 2 \rightarrow 4$
-	Subtraction	$5 - 2 \rightarrow 3$
*	Multiplication	$2 * 2 \rightarrow 4$
/	Division	$2 / 2 \rightarrow 1$
//	Floor division	$5 // 3 \rightarrow 1$
%	Modulus (remainder)	$5 \% 3 \rightarrow 2$

Lists vs Tuples

December 30, 2022

1 Lists vs Tuples

[Explaining Tuples in Python](#)

[Zip Function in Python](#)

[Finding the Index in Python](#)

[List Comprehension in Python](#)

[Enumerate Function in Python](#)

2 List

List - Generic data structure to store a collection of items.

```
[1]: box = []
```

```
[2]: type(box)
```

```
[2]: list
```

When you think about a structure the most important question is what behavior you want to use on that structure. You do not want just to store information with a list > you want to process information with a list > from left to right.

```
[3]: # help(list)
```

Allows you to store any number of information. List is Versatile - you can add; you can remove; you can iterate over the list.

```
[4]: box.append(1)
      box.append(9)
      box.append(4)
      box.append(5)
      box.append(7)
      box
```

```
[4]: [1, 9, 4, 5, 7]
```

```
[5]: # remove first occurrence of value
box.remove(1)
box
```

```
[5]: [9, 4, 5, 7]
```

```
[6]: # you can sort
box.sort()
box
```

```
[6]: [4, 5, 7, 9]
```

you can index and slice lists

```
[7]: box[0]
```

```
[7]: 4
```

```
[8]: box[-1]
```

```
[8]: 9
```

```
[9]: # [start:stop:step]
box[0:3]
```

```
[9]: [4, 5, 7]
```

```
[10]: box[::-1]
```

```
[10]: [9, 7, 5, 4]
```

LISTS ARE MUTABLE

```
[11]: box[0] = 77
box
```

```
[11]: [77, 5, 7, 9]
```

By the General convention list is Homogeneous to store and deal individually with the sequence of the same kind of stuff. However > the same list can accept objects of various different types.

```
[12]: box[1] = 'apple'
box
```

```
[12]: [77, 'apple', 7, 9]
```

Under the hood > Lists do NOT store data items at all > but rather they store Object References.

1. When lists are created they take copies of the Object References they are given.

2. In the case of Integers and Strings > an object of the appropriate data type is created in the memory and then an Object Reference referring to the object is created and it is the Object Reference that is PUT in the List.

```
[13]: len(box)
```

```
[13]: 4
```

Lists are SIZED > they have a notion/concept of size and items can be passed to this data type.

Lists are meant to be iterated over

```
[14]: for number in box:
      print(number)
```

```
77
apple
7
9
```

3 Tuple

Tuples are the natural data structures for Hetrogeneous sequences. Tuples are used for grouping data.

```
[15]: t = 1,2
      type(t)
```

```
[15]: tuple
```

```
[16]: tt = (1,2)
      type(tt)
```

```
[16]: tuple
```

```
[17]: print("This is our original list ", box)
      tupleBox =tuple(box)
      type(tupleBox)
```

This is our original list [77, 'apple', 7, 9]

```
[17]: tuple
```

Tuples are meant to be accessed by position.

```
[18]: tupleBox[1]
```

```
[18]: 'apple'
```

```
[19]: tupleBox[::-1]
```

```
[19]: (9, 7, 'apple', 77)
```

Tuples are immutable.

```
[20]: tupleBox[1] = 77
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-20-35f6ce367f5f> in <module>()  
----> 1 tupleBox[1] = 77  
  
TypeError: 'tuple' object does not support item assignment
```

```
[21]: len(tupleBox)
```

```
[21]: 4
```

Tuples are faster than lists. tuples are stored in a single block of memory, so creating a new tuple involves at worst a single call to allocate memory. Tuple's immutability means that the interpreter can use a leaner, faster data structure for it, compared to list.

Python has a very powerful tuple assignment feature that allows a tuple of variables on the left of an assignment to be assigned values from a tuple on the right of the assignment.

```
[22]: julia = ("Julia", "Roberts", 1967, "Eat Pray Love", 2010, "Actress", "Atlanta, GA",  
             ↪ "Georgia")  
julia
```

```
[22]: ('Julia',  
      'Roberts',  
      1967,  
      'Eat Pray Love',  
      2010,  
      'Actress',  
      'Atlanta, Georgia')
```

```
[23]: (name, surname, b_year, movie, m_year, profession, b_place) = julia  
julia
```

```
[23]: ('Julia',  
      'Roberts',  
      1967,  
      'Eat Pray Love',  
      2010,  
      'Actress',
```

```
'Atlanta, Georgia')
```

```
[24]: name, movie
```

```
[24]: ('Julia', 'Eat Pray Love')
```

One way to think of tuple assignment is as tuple packing/unpacking.

It is useful for swapping.

```
[25]: a,b = 1,2  
      a,b = b,a  
      a,b
```

```
[25]: (2, 1)
```

Naturally, the number of variables on the left and the number of values on the right have to be the same.

```
[26]: (a, b, c, d) = (1, 2, 3)
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-26-3edeb85aaccb> in <module>()  
----> 1 (a, b, c, d) = (1, 2, 3)  
  
ValueError: not enough values to unpack (expected 4, got 3)
```

You can still iterate tuples.

```
[27]: for item in julia:  
      print(item)
```

```
Julia  
Roberts  
1967  
Eat Pray Love  
2010  
Actress  
Atlanta, Georgia
```

Indexing_and_Slicing

December 30, 2022

1 Indexing and slicing

Video Tutorials:

[Enumerate Function in Python](#)

[Range Function in Python](#)

2 Indexing

Lists and tuples are ordered collections of elements. That means they retain items in order.

Each item in a list or tuple can be accessed by its index.

To understand this concept, imagine an apartment building. Tennent Mark lives in apartment number 1 and his neighbor John occupies apartment 2. Mary resides in apartment number 3 and so on. If I want to send a letter to Mark, I will need to include his apartment number in address. The letter would be delivered to Mark since he occupies apartment 1. Later Mark moves to another building and my letter if it was late would be deliver to another tennent currently living in the apartment number 1. The point is no matter who lives in the apartment you can always reach out to that person by the apartment number and every building has apartment number 1, 2 and 3. That is why sometimes local businesses would send their flyers addressed to “current resident” of apartment number 1 or 2 or whatever, hoping to pick up clients no matter who occupies them. Our example can be translated to Python. We have the list stored with the variable building. The list contains names of building residents.

```
[1]: building = ["Mark", "John", "Mary"]
```

```
[2]: building[0]
```

```
[2]: 'Mark'
```

building[0]statement get us “Mark”. Consequently building[1] will retrieve “John” and building[2] expectedly “Mary”,

```
[3]: building[1]
```

```
[3]: 'John'
```

```
[4]: building[2]
```

```
[4]: 'Mary'
```

Indexing concept could be applied to a string. All characters in a string have index. You can try to grab the first character of “apple” by index 0.

```
[5]: word = "apple"
```

```
[6]: word[0]
```

```
[6]: 'a'
```

3 Slicing

Indexing helps us to grab one element or character at a time. What if we need to get two or more? Then we need to slice the sequence. The syntax is quite simple we need to provide indexes for locations of characters where we want to start and where we want to stop.

`object[ start_index : stop_index ]`

word = “APPLE”

word[start index : stop index : step index]

stop index is exclusive

step index is optional default value is **1**

Example:

word[1:3]

Result:

“pp”

For example, if we want to grab “pp” out of “apple” we would need to indicate the index for the first “p”, which is 1, and index for the second “p” plus one, 2+1. I know it sounds a bit confusing

at first. Remember in Python stop point is always excluded. For that reason, we would need to add one to the index of the last character of the sliced substring.

```
[7]: word[ 1:3 ]
```

```
[7]: 'pp'
```

object[start : stop : step]

Step index indicates, intervals, how you want to move through the sequence. By default, step index is 1.

Using the defaulted step index like this

```
[8]: word[0 : : 1]
```

```
[8]: 'apple'
```

would not do much and return the whole string “apple”. The above notation simply stating that we want to start at the first character and then move one by one letter. Skipping the stop index in the middle means that we do not want to stop anywhere and get all character no matter how many is there.

```
[9]: word[::2]
```

```
[9]: 'ape'
```

In the event that you need to skip a character you can use 2 as the step index. word[0 : : 2] statement would return you “ape”. Step index 2 is skipping every other letter.

Step index could be negative. To reverse the whole word and read it backwards we would need to use -1 index as a step.

```
[10]: word[-1 : : -1]
```

```
[10]: 'elppa'
```

The first negative index -1 indicates that we want to start at the last character “e”. The step index -1 means that we want to get all characters from right to left.

If Else statements

December 30, 2022

1 Decision Structures

2 if elif else

Video Tutorials:

[If-Else Statements in Python](#)

[Solving FizzBuzz Problems in Python](#)

```
[2]: temperature = float(input('What is the temperature? '))
```

What is the temperature? 90

```
[4]: # Code blocks are defined by their indentation  
# All statements with the same distance to the right  
# belong to the same block of code  
if temperature > 70:  
    print('Wear shorts.')  
else:  
    print('Wear long pants.')
```

Wear shorts.

Relational operators

< less than
<= less than or equal to
== equal to
>= greater than or equal to
> greater than
!= not equal to

```
[3]: temperature = float(input('What is the temperature? '))  
if temperature > 70:  
    print('Wear shorts.')  
elif temperature == 70:  
    print("Go either way")  
else:  
    print('Wear long pants.')
```

What is the temperature? 70

Go either way

If, elif and else are keywords in Python.

A condition is a test for something (is x less than y, is x == y etc.)

The colon (:) at the end of the if line is required.

Statements are instructions to follow if the condition is true.

These statements must be indented and is only being run when the if condition is met.

If vs Elif.

Multiple if's means your code would go and check all the if conditions, where as in case of elif, if one if condition satisfies it would not check other conditions.

```
[7]: age =18

if age < 21:
    print ("You are a child")
if age > 21:
    print ("You are an adult")
else:    #Handle all cases were 'age' is negative
    print ("The age must be a positive integer!")
```

You are a child

The age must be a positive integer!

Here you can see that when 18 is used as input the answer is (surprisingly) 2 sentences. That is wrong. It should only be the first sentence.

That is because BOTH if statements are being evaluated. The computer sees them as two separate statements:

The first one is true for 18 and so “You are a child” is printed. The second if statement is false and therefore the else part is executed printing “The age must be a positive integer”. The elif fixes this and makes the two if statements ‘stick together’ as one:

```
[5]: age =18

if age < 21:
    print ("You are a child")
elif age > 21:
    print ("You are an adult")
else:    #Handle all cases were 'age' is negative
    print ("The age must be a positive integer!")
```

You are a child

3 Max of Three example

Suppose we need an algorithm to find the largest of three numbers.

```
[8]: x1, x2, x3 = int(input("please enter x1: ")), int(input("and x2: ")),  
    ↪ int(input("and x3: "))  
    print(x1)
```

```
please enter x1: 5  
and x2: 3  
and x3: 2  
5
```

```
[ ]: # x, y = [int(x) for x in input().split()]  
    # print(x)
```

Strategy 1. Compare Each to All.

Let's consider the first probability, that x1 is the largest.

```
[ ]: if x1 >= x2 >= x3:  
    maxval = x1  
    print("True")  
else:  
    print("False")
```

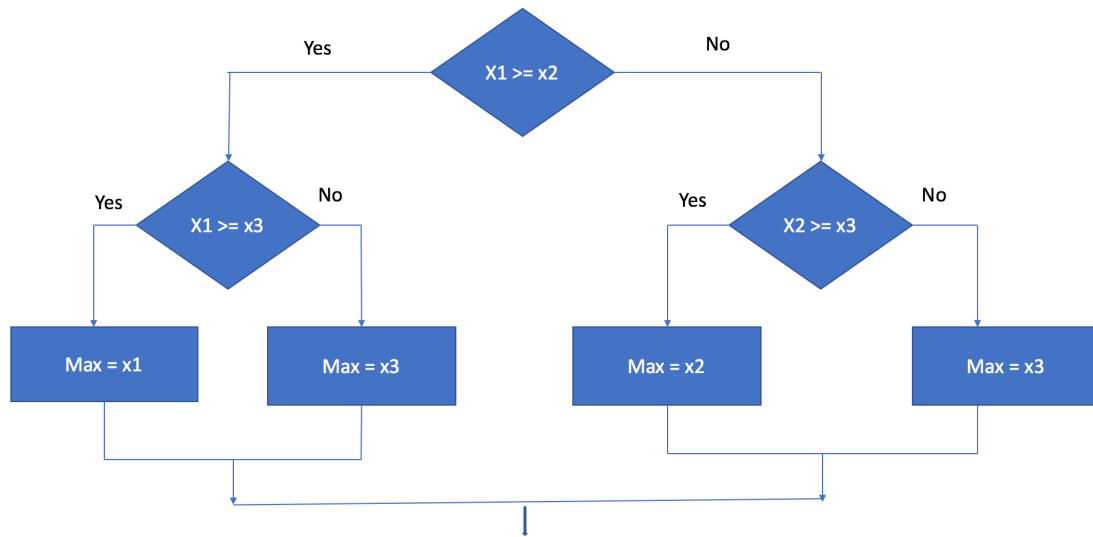
Are we certain that this condition is True in all cases where x1 is the max? What if x1=5, x2=2, x3=4?

Python allows us to test multiple conditions by combining them with the keyword and.

```
[ ]: if x1 >= x2 and x1 >= x3:  
    maxval = x1  
elif x2 >= x1 and x2 >= x3:  
    maxval = x2  
else:  
    maxval = x3  
print(maxval)
```

Strategy 2. Decision Tree.

Flowchart of the decision tree approach



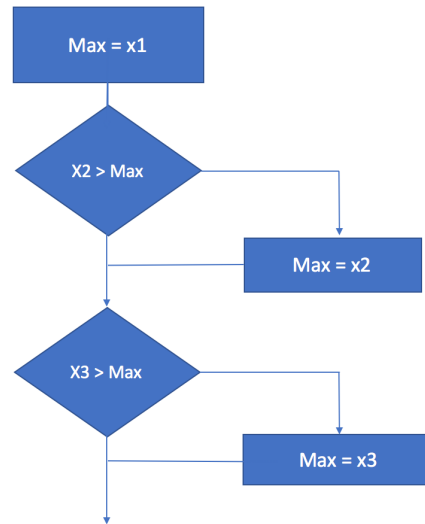
```
[ ]: if x1 >= x2:
    if x1 >= x3:
        maxval = x1
    else:
        maxval = x3
else:
    if x2 > x3:
        maxval = x2
    else:
        maxval = x3

print(maxval)
```

The strength of this approach is its efficiency, No matter what the ordering of the three values, this algorithm will make exactly two comparisons and assign the correct value to maxval.

Strategy 3. Sequential Processing.

Flowchart of a sequential approach



Sequential approach is to scan through the numbers until you find a big one.

```
[ ]: maxval = x1
    if x2 > maxval:
        maxval = x2
    if x3 > maxval:
        maxval = x3
    print(maxval)
```

Clearly, the sequential approach is the best of our three algorithms.

Definite loops

December 30, 2022

1 Definite Loops (For loop)

Video Tutorials:

[Using For Loop in Python](#)

[Range Function in Python](#)

The simplest kind of loop is called a definite loop. This is a loop that will execute a sequence of statements a definite number of times.

A Python for loop has this general form:

```
for <var> in <sequence>:  
    <body>
```

You can iterate through a list

```
[1]: for number in [0,1,2,4,5,6,7]:  
      print(number)
```

0
1
2
4
5
6
7

You can iterate through a string

```
[1]: word ="apple"  
  
for letter in word:  
    print(letter)
```

a
p
p
l
e

You can iterate through a list of strings

```
[3]: shopping_list = ["apples", "milk", "bananas", "bread"]
     for item in shopping_list:
         print(item)
```

```
apples
milk
bananas
bread
```

```
[4]: for item in shopping_list:
     length = len(item)
     print(item, "is", length, "letters")
```

```
apples is 6 letters
milk is 4 letters
bananas is 7 letters
bread is 5 letters
```

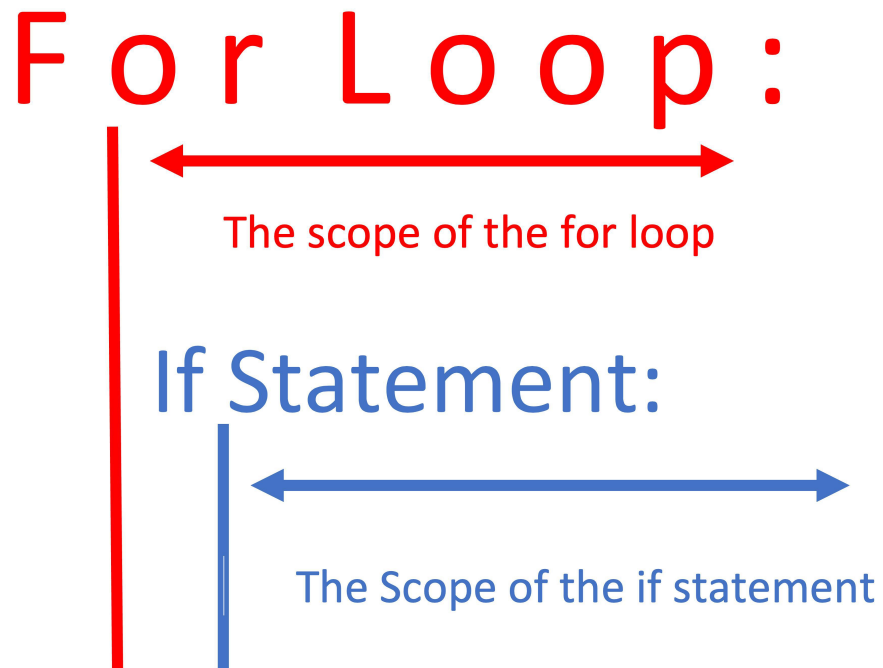
Use if and else in for loop

```
[8]: friend = "dog"

     for l in friend:
         if l == "o":
             print("this is a vowel")
         else:
             print("just a letter")
```

```
just a letter
this is a vowel
just a letter
```


2 Scope of a for loop



3 Function range()

The `range()` function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

Optional. An integer specifying at which number to start. Default start is 0.

Optional. A number specifying the step. Default step is 1.

`range(start : stop : step)`

Required. An integer specifying where to stop. The stop number is not included in generated range of numbers.

```
[7]: for number in range(1,11):  
      print(number)
```

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

You can generate index for all items in a list

```
[6]: for index in range(len(shopping_list)):  
      print(index, shopping_list[index])
```

```
0 apples  
1 milk  
2 bananas  
3 bread
```

```
[ ]:
```

Lambda Operator

December 30, 2022

1 Lambda Operator

Video Tutorials:

[lambda operator in Python](#)

The lambda operator or lambda function is a way to create small anonymous functions.

A Lambda Function , or a Small Anonymous Function , is a self-contained block of functionality that can be passed around and used in your code.

The general syntax of a lambda function is quite simple:

`lambda arguments: expression`

The argument list consists of a comma separated list of arguments and the expression is an arithmetic expression using these arguments. You can assign the function to a variable to give it a name.

IN **out**

lambda variable : expression

```
[ ]: f = lambda x, y : x + y  
      f(2,3)
```

```
[ ]: def g(x,y): return x + y  
      g(2,3)
```

As you can see, f() and g() do exactly the same and can be used in the same ways.

Note that the lambda definition does not include a “return” statement – it always contains an expression which is returned.

Also note that you can put a lambda definition anywhere a function is expected, and you don't have to assign it to a variable at all.

map() function

The map() function applies a given function to each item of an iterable (list, tuple etc.) and returns a list of the results.

```
[ ]: def calculateSquare(n):  
      return n*n  
  
numbers = (1, 2, 3, 4)  
result = map(calculateSquare, numbers)  
list(result)
```

How to use lambda function with map()

```
[ ]: numbers = (1, 2, 3, 4)  
result = map(lambda x: x*x, numbers)  
list(result)
```

Passing Multiple Iterators to map() Using Lambda

```
[5]: box1 = [4, 5, 6]  
      box2 = [5, 6, 7]  
  
result = map(lambda orange, banana: orange+banana, box1, box2)  
list(result)
```

```
[5]: [9, 11, 13]
```

filter function

filter(function,iterable) creates a new list from the elements for which the function returns True

```
[4]: alist = list(range(1,100))  
      newList = filter(lambda x: x % 3 == 0 and x % 5 == 0 , alist)  
      # newList  
      list(newList)
```

```
[4]: [15, 30, 45, 60, 75, 90]
```

Those things are actually quite useful. Python supports a style of programming called functional programming where you can pass functions to other functions to do stuff.

```
[2]: alist = ["MON","TUE","WED","THU","FRI","SAT","SUN"]  
  
list(map(lambda x: x.lower(),alist))
```

```
[2]: ['mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun']
```

```
[6]: [x.capitalize() for x in alist]
```

```
[6]: ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
```

```
[8]: newlist = [[567,345,234],[253,465,756, 2345],[333,777,111, 555]]
```

```
[9]: list(map(lambda x: sorted(x), newlist))
```

```
[9]: [[234, 345, 567], [253, 465, 756, 2345], [111, 333, 555, 777]]
```

2 Lambda with if and else

do this if True If True If False, do something else

lambda variable : *if-expression* *if condition* *else else-expression*

```
[3]: numbers = list(range(1,11))
```

```
[4]: numbers
```

```
[4]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
[7]: new = list(map(lambda n : "Even" if n % 2 == 0 else "Odd", numbers))
```

```
[8]: new
```

```
[8]: ['Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even']
```

```
[ ]:
```

Pandas DataFrame

December 30, 2022

1 The Pandas DataFrame Object

Video Tutorials:

[Dictionary Data Type in Python](#)

[DataFrames in Pandas](#)

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

The DataFrame can be thought of either a generalization of NumPy array, or as specialization of a Python dictionary. DataFrame is a two-dimensional array.

```
[1]: import pandas as pd
```

Create Data Frame object

```
[2]: stocks = ["IBM", "APPLE", "TWTR", "GE", "MSFT"]
     prices = [115.00, 119.14, 19.77, 25.99, 26]
```

```
[3]: portfolio = list(zip(stocks,prices))
     portfolio
```

```
[3]: [('IBM', 115.0),
      ('APPLE', 119.14),
      ('TWTR', 19.77),
      ('GE', 25.99),
      ('MSFT', 26)]
```

```
[4]: df = pd.DataFrame(data=portfolio, columns=['Stocks', 'Prices'])
     df
```

```
[4]:   Stocks  Prices
0    IBM   115.00
1  APPLE   119.14
2  TWTR    19.77
3    GE    25.99
4  MSFT    26.00
```

DataFrame has three attributes: values, index and columns

```
[5]: df.values
```

```
[5]: array([[ 'IBM', 115.0],
          [ 'APPLE', 119.14],
          [ 'TWTR', 19.77],
          [ 'GE', 25.99],
          [ 'MSFT', 26.0]], dtype=object)
```

```
[6]: df.index
```

```
[6]: RangeIndex(start=0, stop=5, step=1)
```

```
[7]: df.columns
```

```
[7]: Index(['Stocks', 'Prices'], dtype='object')
```

You could always add another column

```
[8]: df["Volume"] = 0
df
```

```
[8]:   Stocks  Prices  Volume
0    IBM    115.00      0
1  APPLE    119.14      0
2  TWTR     19.77      0
3     GE     25.99      0
4   MSFT     26.00      0
```

```
[9]: df["Volume"] = [40,36,23,44,25]
df
```

```
[9]:   Stocks  Prices  Volume
0    IBM    115.00     40
1  APPLE    119.14     36
2  TWTR     19.77     23
3     GE     25.99     44
4   MSFT     26.00     25
```

We can change the index

```
[11]: df.index = ['a','b','c','d','e']
df
```

```
[11]:   Stocks  Prices  Volume
a    IBM    115.00     40
b  APPLE    119.14     36
c  TWTR     19.77     23
d     GE     25.99     44
```

```
e    MSFT    26.00    25
```

The DataFrame can be thought of as a generalization of a two-dimensional NumPy array, where both the rows and columns have a generalized index for accessing the data.

Slicing DataFrame objects

```
[12]: df.Stocks
```

```
[12]: a      IBM
      b    APPLE
      c    TWTR
      d      GE
      e    MSFT
      Name: Stocks, dtype: object
```

Slicing a column with square brackets

```
[13]: df["Stocks"]
```

```
[13]: a      IBM
      b    APPLE
      c    TWTR
      d      GE
      e    MSFT
      Name: Stocks, dtype: object
```

Indexing

Indexing using square brackets, pass column label and index

```
[14]: df["Stocks"]['c']
```

```
[14]: 'TWTR'
```

A column is an attribute of the DataFrame

```
[15]: df.Stocks['c']
```

```
[15]: 'TWTR'
```

DataFrame also has the loc and iloc attributes

Indexing by labels done with method .loc

```
[16]: df.loc['c']
```

```
[16]: Stocks    TWTR
      Prices    19.77
      Volume    23
```


Name: c, dtype: object

```
[17]: df.loc['c', 'Prices']
```

```
[17]: 19.77
```

iloc method - indexing by index positions, pass numerical indices of row and column

```
[18]: print(df)
df.iloc[4,1]
```

	Stocks	Prices	Volume
a	IBM	115.00	40
b	APPLE	119.14	36
c	TWTR	19.77	23
d	GE	25.99	44
e	MSFT	26.00	25

```
[18]: 26.0
```

Pandas DataFrame advanced operations

December 30, 2022

1 Advanced Operations on DataFrame

DataFrame is a specialization of a dictionary. Where a dictionary maps a key to a value, a DataFrame maps name to a Series of column data.

A Pandas DataFrame can be constructed in a variety of ways.

```
[14]: import pandas as pd
import numpy as np
```

```
[2]: cities = ["Philadelphia", "Boston", "Baltimore", "Orlando"]
states = ["PA", "MA", "MD", "FL"]
population = [1.5, 0.6, 0.6, 2.35]
visitors = [41, 19, 24, 66]
```

Create a list of labels

```
[3]: list_labels = ["City", "State", "Population", "Visitors"]
list_cols = [cities, states, population, visitors]
```

```
[4]: zipped = list(zip(list_labels, list_cols))
zipped
```

```
[4]: [('City', ['Philadelphia', 'Boston', 'Baltimore', 'Orlando']),
      ('State', ['PA', 'MA', 'MD', 'FL']),
      ('Population', [1.5, 0.6, 0.6, 2.35]),
      ('Visitors', [41, 19, 24, 66])]
```

Creating a dictionary

```
[5]: data = dict(zipped)
data
```

```
[5]: {'City': ['Philadelphia', 'Boston', 'Baltimore', 'Orlando'],
      'State': ['PA', 'MA', 'MD', 'FL'],
      'Population': [1.5, 0.6, 0.6, 2.35],
      'Visitors': [41, 19, 24, 66]}
```

```
[6]: df = pd.DataFrame(data)
df
```

```
[6]:
```

	City	State	Population	Visitors
0	Philadelphia	PA	1.50	41
1	Boston	MA	0.60	19
2	Baltimore	MD	0.60	24
3	Orlando	FL	2.35	66

Reassign the index

```
[7]: df.index = df["State"]
df
```

```
[7]:
```

	City	State	Population	Visitors
State				
PA	Philadelphia	PA	1.50	41
MA	Boston	MA	0.60	19
MD	Baltimore	MD	0.60	24
FL	Orlando	FL	2.35	66

Drop a column

axis : {0 or 'index', 1 or 'columns'}, default 0
Whether to drop labels from the index (0 or 'index') or columns (1 or 'columns').

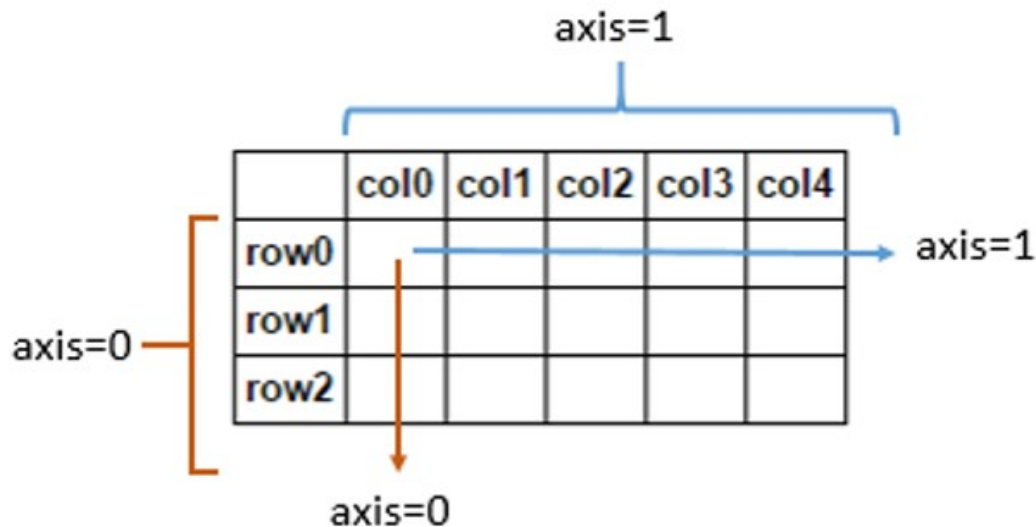
axis = 1 denotes that we are referring to a column, not a row

```
[8]: df = df.drop('State', axis=1)
df
```

```
[8]:
```

	City	Population	Visitors
State			
PA	Philadelphia	1.50	41
MA	Boston	0.60	19
MD	Baltimore	0.60	24
FL	Orlando	2.35	66

axis means whether to drop labels from the index (0 / 'index') or columns (1 / 'columns').



We can change a specified column name in a DataFrame

```
[13]: df.rename(columns={'Visitors': 'Number_Visitors'}, inplace=True)
df
```

```
[13]:
```

	City	Population	Number_Visitors
State			
PA	Philadelphia	1.50	41
MA	Boston	0.60	19
MD	Baltimore	0.60	24
FL	Orlando	2.35	66

2 Universal Functions

Pandas inherits much of functionality from NumPy. Pandas is design to work with NumPy, any NumPy Ufunc will work on Pandas Series and DataFrame objects.

```
[16]: np.round(df.Population.sum())
```

```
[16]: 5.0
```

```
[17]: df.Number_Visitors.sum()
```

```
[17]: 150
```

another example np.min(), find min in your column

```
[18]: np.min(df.Number_Visitors)
```

```
[18]: 19
```

since we think of a DataFrame as a specialization of a dictionary, we can use dictionary-like Python expressions and methods.

```
[19]: df.keys()
```

```
[19]: Index(['City', 'Population', 'Number_Visitors'], dtype='object')
```

```
[20]: dict(df.items())
```

```
[20]: {'City': State
      PA    Philadelphia
      MA         Boston
      MD    Baltimore
      FL    Orlando
      Name: City, dtype: object,
      'Population': State
      PA    1.50
      MA    0.60
      MD    0.60
      FL    2.35
      Name: Population, dtype: float64,
      'Number_Visitors': State
      PA    41
      MA    19
      MD    24
      FL    66
      Name: Number_Visitors, dtype: int64}
```

```
[21]: list(df.items())
```

```
[21]: [('City',
      State
      PA    Philadelphia
      MA         Boston
      MD    Baltimore
      FL    Orlando
      Name: City, dtype: object),
      ('Population',
      State
      PA    1.50
      MA    0.60
      MD    0.60
      FL    2.35
      Name: Population, dtype: float64),
      ('Number_Visitors',
      State
      PA    41
      MA    19
```

```
MD      24
FL      66
Name: Number_Visitors, dtype: int64)]
```

3 DataFrame Best practica is to use apply method NOT for loop

The apply() method lets you apply an arbitrary function to the group results or a column. The apply() method returns either a Pandas object or a scalar.

apply() is the most efficient way to do it especially for a big data set

```
[22]: df["new_city_length"] = df["City"].apply(len)
df
```

```
[22]:
```

	City	Population	Number_Visitors	new_city_length
State				
PA	Philadelphia	1.50	41	12
MA	Boston	0.60	19	6
MD	Baltimore	0.60	24	9
FL	Orlando	2.35	66	7

We can create and apply custom functions in Pandas

```
[24]: def hundreds(n):
      return n/100

df["Number_Visitors"].apply(hundreds)
```

```
[24]: State
PA      0.41
MA      0.19
MD      0.24
FL      0.66
Name: Number_Visitors, dtype: float64
```

We can save this data in a new column

```
[26]: df["Visitors/hundreds"] = df.Number_Visitors.apply(lambda n: n/100)
df
```

```
[26]:
```

	City	Population	Number_Visitors	new_city_length	\
State					
PA	Philadelphia	1.50	41	12	
MA	Boston	0.60	19	6	
MD	Baltimore	0.60	24	9	
FL	Orlando	2.35	66	7	

Visitors/hundreds

State	
PA	0.41
MA	0.19
MD	0.24
FL	0.66

4 Filtering DataFrame

Create custom filters

```
[27]: greater_than = df.new_city_length > 7
df[greater_than]
```

```
[27]:
```

	City	Population	Number_Visitors	new_city_length	\
State					
PA	Philadelphia	1.5	41	12	
MD	Baltimore	0.6	24	9	

Visitors/hundreds

State	
PA	0.41
MD	0.24

or we could do

```
[28]: df[df.new_city_length > 7]
```

```
[28]:
```

	City	Population	Number_Visitors	new_city_length	\
State					
PA	Philadelphia	1.5	41	12	
MD	Baltimore	0.6	24	9	

Visitors/hundreds

State	
PA	0.41
MD	0.24

we can combine 2 filters using the following syntax

```
[29]: df[(df.new_city_length > 7) & (df.new_city_length < 10)]
```

```
[29]:
```

	City	Population	Number_Visitors	new_city_length	\
State					
MD	Baltimore	0.6	24	9	

Visitors/hundreds

State	
MD	0.24

```
[30]: df[(df.new_city_length == 9) | (df.new_city_length >= 10)]
```

```
[30]:
```

	City	Population	Number_Visitors	new_city_length	\
State					
PA	Philadelphia	1.5	41	12	
MD	Baltimore	0.6	24	9	

	Visitors/hundreds
State	
PA	0.41
MD	0.24

We can filter one column based on another

```
[31]: df.City[df.new_city_length == 7]
```

```
[31]: State  
FL    Orlando  
Name: City, dtype: object
```


Pandas Cleaning and Manipulating Data

December 30, 2022

1 Pandas Cleaning and Manipulating Data

```
[20]: import pandas as pd
import matplotlib.pyplot as plt
```

Using read_csv method read data from remote location

Use skiprows=1 to get rid of the first row in the dataset

Use method head to get first 5 rows of data

```
[12]: url = 'https://www.sectorspdr.com/sectorspdr/IDCO.Client.Spdrs.Index/Export/
↳ExportCsv?symbol=xlf'
df = pd.read_csv(url, skiprows=1)
df.head()
```

```
[12]:
```

	Symbol	Company Name	Index	Weight	Last	Change	%Change	\
0	AFL	AFLAC Inc		1.09%	72.16	0.69	+0.97%	
1	AIG	American Intl Group Inc		1.26%	63.65	0.76	+1.21%	
2	AIZ	Assurant Inc		0.18%	125.51	1.55	+1.25%	
3	ALL	Allstate Corp		0.97%	136.60	1.17	+0.86%	
4	AMP	Ameriprise Financial Inc		0.87%	311.12	6.57	+2.16%	

	Volume	52 Week Range	Unnamed: 8
0	458.52 K	52.07 - 72.70	NaN
1	511.46 K	47.05 - 65.73	NaN
2	90.62 K	119.01 - 194.12	NaN
3	201.65 K	111.85 - 144.46	NaN
4	114.48 K	219.99 - 339.41	NaN

Clean dataset and drop 'Unnamed: 8' column

```
[13]: df.drop('Unnamed: 8', axis=1, inplace=True)
new_file.head()
```

```
[13]:
```

	Symbol	Company Name	Index	Weight	Last	Change	%Change	\
0	AFL	AFLAC Inc		1.09%	72.11	0.64	+0.90%	
1	AIG	American Intl Group Inc		1.26%	63.64	0.75	+1.19%	
2	AIZ	Assurant Inc		0.18%	125.47	1.51	+1.22%	

3	ALL	Allstate Corp	0.97%	136.42	0.99	+0.73%
4	AMP	Ameriprise Financial Inc	0.87%	311.16	6.61	+2.17%

	Volume	52 Week Range	Unnamed: 8
0	453.90 K	52.07 - 72.70	NaN
1	506.53 K	47.05 - 65.73	NaN
2	88.02 K	119.01 - 194.12	NaN
3	199.91 K	111.85 - 144.46	NaN
4	111.98 K	219.99 - 339.41	NaN

Reassign index of the DataFrame to values from the column Symbol and drop the column Symbol

```
[14]: df.index = df.Symbol
df.drop('Symbol', axis=1, inplace=True)
df.head()
```

```
[14]:
```

	Company Name	Index Weight	Last	Change	%Change	\
Symbol						
AFL	AFLAC Inc	1.09%	72.16	0.69	+0.97%	
AIG	American Intl Group Inc	1.26%	63.65	0.76	+1.21%	
AIZ	Assurant Inc	0.18%	125.51	1.55	+1.25%	
ALL	Allstate Corp	0.97%	136.60	1.17	+0.86%	
AMP	Ameriprise Financial Inc	0.87%	311.12	6.57	+2.16%	

	Volume	52 Week Range
Symbol		
AFL	458.52 K	52.07 - 72.70
AIG	511.46 K	47.05 - 65.73
AIZ	90.62 K	119.01 - 194.12
ALL	201.65 K	111.85 - 144.46
AMP	114.48 K	219.99 - 339.41

Drop a 'JPM' row from the dataset

```
[15]: df.drop("JPM", inplace=True)
df.head()
```

```
[15]:
```

	Company Name	Index Weight	Last	Change	%Change	\
Symbol						
AFL	AFLAC Inc	1.09%	72.16	0.69	+0.97%	
AIG	American Intl Group Inc	1.26%	63.65	0.76	+1.21%	
AIZ	Assurant Inc	0.18%	125.51	1.55	+1.25%	
ALL	Allstate Corp	0.97%	136.60	1.17	+0.86%	
AMP	Ameriprise Financial Inc	0.87%	311.12	6.57	+2.16%	

	Volume	52 Week Range
Symbol		
AFL	458.52 K	52.07 - 72.70

```

AIG      511.46 K    47.05 - 65.73
AIZ       90.62 K   119.01 - 194.12
ALL      201.65 K   111.85 - 144.46
AMP      114.48 K   219.99 - 339.41

```

Print all column names

```
[16]: df.columns
```

```
[16]: Index(['Company Name', 'Index Weight', 'Last', 'Change', '%Change', 'Volume',
          '52 Week Range'],
          dtype='object')
```

We can get additional info about our dataset using method info()

```
[17]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 66 entries, AFL to ACGL
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Company Name    66 non-null    object
 1   Index Weight    66 non-null    object
 2   Last            66 non-null    float64
 3   Change          66 non-null    float64
 4   %Change         66 non-null    object
 5   Volume          66 non-null    object
 6   52 Week Range   66 non-null    object
dtypes: float64(2), object(5)
memory usage: 4.1+ KB

```

Count all unique values

```
[18]: df.Change.value_counts().head(10)
```

```

[18]: 1.81      2
      0.94      2
      0.53      2
      0.69      1
      2.43      1
      0.38      1
      1.37      1
      3.13      1
      1.78      1
      0.73      1
      Name: Change, dtype: int64

```

Get all statistical measurements using method describe()

```
[19]: df.describe()
```

```
[19]:
```

	Last	Change
count	66.000000	66.000000
mean	139.312727	2.844394
std	127.498984	3.851843
min	14.140000	0.160000
25%	58.175000	0.835000
50%	98.755000	1.375000
75%	169.447500	2.755000
max	717.710000	18.190000

Get all the companies with values in the Change column greater then 0.5

```
[26]: df[df.Change > 0.5].head()
```

```
[26]:
```

	Company Name	Index Weight	Last	Change	%Change	\
Symbol						
AFL	AFLAC Inc	1.09%	72.16	0.69	+0.97%	
AIG	American Intl Group Inc	1.26%	63.65	0.76	+1.21%	
AIZ	Assurant Inc	0.18%	125.51	1.55	+1.25%	
ALL	Allstate Corp	0.97%	136.60	1.17	+0.86%	
AMP	Ameriprise Financial Inc	0.87%	311.12	6.57	+2.16%	

	Volume	52 Week Range
Symbol		
AFL	458.52 K	52.07 - 72.70
AIG	511.46 K	47.05 - 65.73
AIZ	90.62 K	119.01 - 194.12
ALL	201.65 K	111.85 - 144.46
AMP	114.48 K	219.99 - 339.41

Identify all companies where Volume is in thouthands

```
[27]: df[df['Volume'].str.contains("K")]
```

```
[27]:
```

	Company Name	Index Weight	Last	Change	%Change	\
Symbol						
AFL	AFLAC Inc	1.09%	72.16	0.69	+0.97%	
AIG	American Intl Group Inc	1.26%	63.65	0.76	+1.21%	
AIZ	Assurant Inc	0.18%	125.51	1.55	+1.25%	
ALL	Allstate Corp	0.97%	136.60	1.17	+0.86%	
AMP	Ameriprise Financial Inc	0.87%	311.12	6.57	+2.16%	
AON	Aon plc	1.67%	304.36	4.85	+1.62%	
AXP	American Express Co	2.32%	147.31	3.30	+2.29%	
BLK	BlackRock Inc	2.84%	717.71	17.38	+2.48%	
CB	Chubb Limited	2.46%	222.88	3.06	+1.39%	
CINF	Cincinnati Financial Corp	0.44%	104.67	1.96	+1.91%	

CMA	Comerica Inc (MI)	0.23%	67.07	1.44	+2.19%
CME	CME Group Inc A	1.63%	170.16	2.22	+1.32%
COF	Capital One Financial	0.92%	91.96	2.26	+2.51%
DFS	Discover Financial Services	0.70%	97.33	2.05	+2.15%
GS	Goldman Sachs Group Inc	3.12%	343.64	2.77	+0.81%
HIG	Hartford Finl Services Group	0.65%	76.51	0.83	+1.10%
ICE	Intercontinental Exchange Inc	1.54%	104.35	2.08	+2.03%
L	Loews Corp	0.31%	58.82	0.94	+1.62%
LNC	Lincoln National Corp	0.12%	30.50	0.97	+3.28%
MCO	Moody's Corp	1.16%	282.01	8.99	+3.29%
MET	Metlife Inc	1.28%	72.91	1.10	+1.53%
MMC	Marsh & McLennan Companies	2.20%	167.31	2.71	+1.64%
MTB	M&T Bank Corp	0.67%	145.54	1.97	+1.37%
NDAQ	Nasdaq Inc	0.56%	62.01	1.31	+2.16%
NTRS	Northern Trust Corp (IL)	0.49%	89.49	2.43	+2.79%
PGR	Progressive Corp	2.05%	131.06	1.34	+1.03%
PNC	PNC Finl Services Group	1.70%	157.67	1.81	+1.16%
PRU	Prudential Financial Inc	0.98%	100.18	1.81	+1.84%
STT	State Street Corp	0.76%	78.21	1.37	+1.78%
TROW	T Rowe Price Group Inc	0.65%	111.21	3.13	+2.90%
TRV	Travelers Cos Inc	1.19%	189.91	1.78	+0.95%
ZION	Zions Bancorporation N.A.	0.19%	49.27	1.30	+2.71%
CFG	Citizens Financial Group Inc	0.52%	39.54	0.57	+1.45%
SPGI	S&P Global Inc	2.96%	339.00	9.40	+2.85%
AJG	Arthur J. Gallagher & Co	1.07%	190.53	2.58	+1.37%
RJF	Raymond James Financial Inc	0.56%	107.49	1.23	+1.16%
RE	Everest Re Group Ltd	0.35%	335.33	1.38	+0.41%
PFG	Principal Financial Group	0.52%	85.61	1.21	+1.43%
SIVB	SVB Financial Group	0.35%	234.64	18.19	+8.40%
MSCI	MSCI Inc	0.98%	470.70	16.26	+3.58%
FRC	First Republic Bank	0.59%	122.73	3.96	+3.33%
MKTX	MarketAxess Holdings	0.28%	284.68	8.23	+2.98%
GL	Globe Life Inc.	0.29%	121.08	1.24	+1.03%
WRB	WR Berkley Corp	0.40%	73.65	0.85	+1.17%
CBOE	Cboe Global Markets, Inc	0.36%	127.08	0.96	+0.76%
BRO	Brown & Brown Inc	0.36%	57.96	1.39	+2.46%
FDS	FactSet Research System Inc	0.41%	403.80	9.28	+2.35%
SBNY	Signature Bank NY	0.19%	116.23	4.54	+4.06%
WTW	Willis Towers Watson PLC	0.71%	245.54	4.62	+1.92%
ACGL	Arch Capital Group Ltd	0.62%	63.26	0.66	+1.05%

	Volume	52 Week Range
Symbol		
AFL	458.52 K	52.07 - 72.70
AIG	511.46 K	47.05 - 65.73
AIZ	90.62 K	119.01 - 194.12
ALL	201.65 K	111.85 - 144.46

AMP	114.48 K	219.99 - 339.41
AON	127.22 K	246.21 - 341.98
AXP	660.99 K	130.65 - 199.55
BLK	182.07 K	503.12 - 927.48
CB	360.44 K	173.78 - 222.33
CINF	99.03 K	88.66 - 143.22
CMA	219.46 K	62.83 - 102.09
CME	745.86 K	166.58 - 256.94
COF	638.97 K	86.98 - 162.40
DFS	534.31 K	87.64 - 130.81
GS	447.79 K	277.84 - 412.66
HIG	234.16 K	60.17 - 76.97
ICE	378.78 K	88.60 - 138.46
L	166.66 K	49.36 - 68.20
LNC	632.73 K	28.61 - 76.40
MCO	94.30 K	230.16 - 401.49
MET	546.37 K	57.41 - 77.36
MMC	180.14 K	142.80 - 183.14
MTB	217.34 K	138.43 - 193.42
NDAQ	304.90 K	46.77 - 70.55
NTRS	194.81 K	76.15 - 135.15
PGR	278.35 K	100.81 - 134.50
PNC	390.90 K	143.52 - 228.14
PRU	290.46 K	85.46 - 124.22
STT	679.18 K	58.62 - 104.87
TROW	401.61 K	93.53 - 200.90
TRV	176.22 K	149.65 - 191.22
ZION	199.47 K	45.21 - 75.44
CFG	810.46 K	32.66 - 57.00
SPGI	437.47 K	279.32 - 477.49
AJG	145.88 K	147.33 - 201.51
RJF	267.07 K	84.86 - 126.00
RE	37.89 K	244.58 - 340.66
PFG	231.68 K	61.05 - 96.17
SIVB	468.46 K	198.10 - 752.68
MSCI	125.49 K	376.41 - 619.06
FRC	197.19 K	106.86 - 209.33
MKTX	37.95 K	217.44 - 417.08
GL	131.04 K	87.87 - 122.12
WRB	176.51 K	53.34 - 76.99
CBOE	150.48 K	103.82 - 131.19
BRO	267.45 K	52.91 - 74.00
FDS	52.44 K	345.92 - 495.40
SBNY	528.44 K	110.06 - 374.76
WTW	70.11 K	187.89 - 252.99
ACGL	413.55 K	41.05 - 63.88

Convert all values in the column Volume into absolute numbers

```
[28]: def num_k(x):
        if "K" in x:
            x = x.replace('K', ' ')
            x = pd.to_numeric(x, errors='coerce')
            x *= 1000
        else:
            x = x.replace('M', ' ')
            x = pd.to_numeric(x, errors='coerce')
            x *= 1000000
        return x

df['Volume'] = df['Volume'].apply(num_k)
df.head()
```

```
[28]:
```

	Company Name	Index Weight	Last	Change	%Change	\
Symbol						
AFL	AFLAC Inc	1.09%	72.16	0.69	+0.97%	
AIG	American Intl Group Inc	1.26%	63.65	0.76	+1.21%	
AIZ	Assurant Inc	0.18%	125.51	1.55	+1.25%	
ALL	Allstate Corp	0.97%	136.60	1.17	+0.86%	
AMP	Ameriprise Financial Inc	0.87%	311.12	6.57	+2.16%	

	Volume	52 Week Range
Symbol		
AFL	458520.0	52.07 - 72.70
AIG	511460.0	47.05 - 65.73
AIZ	90620.0	119.01 - 194.12
ALL	201650.0	111.85 - 144.46
AMP	114480.0	219.99 - 339.41

```
[30]: df.Volume.sum()
```

```
[30]: 57768810.0
```

Clean values in the column Index Weight and convert them to numeric data types

```
[31]: df["Index Weight"] = df["Index Weight"].str.replace("%", "").apply(pd.
        ↳to_numeric)
df.head()
```

```
[31]:
```

	Company Name	Index Weight	Last	Change	%Change	\
Symbol						
AFL	AFLAC Inc	1.09	72.16	0.69	+0.97%	
AIG	American Intl Group Inc	1.26	63.65	0.76	+1.21%	
AIZ	Assurant Inc	0.18	125.51	1.55	+1.25%	
ALL	Allstate Corp	0.97	136.60	1.17	+0.86%	

AMP	Ameriprise Financial Inc	0.87	311.12	6.57	+2.16%
-----	--------------------------	------	--------	------	--------

	Volume	52 Week Range
Symbol		
AFL	458520.0	52.07 - 72.70
AIG	511460.0	47.05 - 65.73
AIZ	90620.0	119.01 - 194.12
ALL	201650.0	111.85 - 144.46
AMP	114480.0	219.99 - 339.41

Check all the columns containing numeric data types

```
[33]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 66 entries, AFL to ACGL
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Company Name    66 non-null    object
1   Index Weight    66 non-null    float64
2   Last            66 non-null    float64
3   Change          66 non-null    float64
4   %Change         66 non-null    object
5   Volume          66 non-null    float64
6   52 Week Range  66 non-null    object
dtypes: float64(4), object(3)
memory usage: 6.2+ KB
```

```
[ ]:
```


Pandas Combining Datasets Concatenation

December 30, 2022

1 Concatination

```
[2]: import pandas as pd
import numpy as np
```

Concatenation of two different datasets with the `pd.concat` function

Concatenation of Series and DataFrame objects is very similar to concatenation of Numpy arrays `np.concatenate`

```
[3]: x = [1,2,3]
y = [1,2,3]
z = [1,2,3]

np.concatenate([x,y,z])
```

```
[3]: array([1, 2, 3, 1, 2, 3, 1, 2, 3])
```

The first argument is a list of arrays to concatenate, second is an axis keyword that allows you to specify the axis along which the result will be concatenated

```
[4]: x = [[1, 2],
          [3, 4]]
np.concatenate([x, x], axis=1)
```

```
[4]: array([[1, 2, 1, 2],
           [3, 4, 3, 4]])
```

```
[5]: x = [[1, 2],
          [3, 4]]
np.concatenate([x, x], axis=0)
```

```
[5]: array([[1, 2],
           [3, 4],
           [1, 2],
           [3, 4]])
```

Pandas has a function, `pd.concat()`, which has a similar syntax to `np.concatenate`

```
[6]: a = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])
      b = pd.Series(['D', 'E', 'F'], index=[1, 2, 5])
      pd.concat([a, b])
```

```
[6]: 1    A
      2    B
      3    C
      1    D
      2    E
      5    F
      dtype: object
```

Now lets join DataFrames

```
[7]: A1 = pd.DataFrame(np.arange(9).reshape(3, 3),
                       columns=list('BAC'))
      B1 = pd.DataFrame(np.arange(4).reshape(2, 2),
                       columns=list('ED'))
      A1
```

```
[7]:   B  A  C
      0  0  1  2
      1  3  4  5
      2  6  7  8
```

```
[8]: B1
```

```
[8]:   E  D
      0  0  1
      1  2  3
```

Note that integer columns get cast to a float if NaNs are introduced

```
[9]: # pd.concat([A1, B1], axis='columns')
      newDf = pd.concat([A1, B1], axis='rows')
      newDf
```

```
[9]:   B  A  C  E  D
      0  0.0  1.0  2.0  NaN  NaN
      1  3.0  4.0  5.0  NaN  NaN
      2  6.0  7.0  8.0  NaN  NaN
      0  NaN  NaN  NaN  0.0  1.0
      1  NaN  NaN  NaN  2.0  3.0
```

Pandas concatenation preserves indices, even if the result will have duplicate indices

```
[10]: newDf = newDf.reset_index(drop=True)
      newDf
```

```
[10]:
```

	B	A	C	E	D
0	0.0	1.0	2.0	NaN	NaN
1	3.0	4.0	5.0	NaN	NaN
2	6.0	7.0	8.0	NaN	NaN
3	NaN	NaN	NaN	0.0	1.0
4	NaN	NaN	NaN	2.0	3.0

```
[11]: newDf.index.values
```

```
[11]: array([0, 1, 2, 3, 4])
```

Index duplicates

If you'd like to simply verify that the indices in the result of `pd.concat()` do not overlap, you can specify the `verify_integrity` flag. With this set to `True`, the concatenation will raise an exception if there are duplicate indices

```
[12]: try:
        pd.concat([A1, B1], verify_integrity=True)
    except ValueError as e:
        print("ValueError:", e)
```

ValueError: Indexes have overlapping values: Int64Index([0, 1], dtype='int64')

Ignoring the index

```
[13]: pd.concat([A1, B1], ignore_index=True)
```

```
[13]:
```

	B	A	C	E	D
0	0.0	1.0	2.0	NaN	NaN
1	3.0	4.0	5.0	NaN	NaN
2	6.0	7.0	8.0	NaN	NaN
3	NaN	NaN	NaN	0.0	1.0
4	NaN	NaN	NaN	2.0	3.0

Adding MultiIndex keys

Another option is to use the `keys` option to specify a label for the data sources; the result will be a hierarchically indexed series containing the data

```
[14]: pd.concat([A1, B1], keys=['x', 'y'])
```

```
[14]:
```

	B	A	C	E	D
x 0	0.0	1.0	2.0	NaN	NaN
1	3.0	4.0	5.0	NaN	NaN
2	6.0	7.0	8.0	NaN	NaN
y 0	NaN	NaN	NaN	0.0	1.0
1	NaN	NaN	NaN	2.0	3.0

The result is a multi-indexed DataFrame

Concatenation with joins

```
[15]: A = pd.DataFrame(np.arange(9).reshape(3, 3),
                        columns=list('ABC'))
      B = pd.DataFrame(np.arange(9).reshape(3, 3),
                        columns=list('BCD'))
      A
```

```
[15]:   A  B  C
      0  0  1  2
      1  3  4  5
      2  6  7  8
```

```
[16]: B
```

```
[16]:   B  C  D
      0  0  1  2
      1  3  4  5
      2  6  7  8
```

```
[17]: pd.concat([A, B])
```

```
[17]:   A  B  C  D
      0  0.0  1  2  NaN
      1  3.0  4  5  NaN
      2  6.0  7  8  NaN
      0  NaN  0  1  2.0
      1  NaN  3  4  5.0
      2  NaN  6  7  8.0
```

By default, the entries for which no data is available are filled with NA values. To change this, we can specify one of several options for the `join` and `join_axes` parameters of the `concatenate` function. By default, the join is a union of the input columns (`join='outer'`), but we can change this to an intersection of the columns using `join='inner'`

```
[18]: pd.concat([A, B], join='inner')
```

```
[18]:   B  C
      0  1  2
      1  4  5
      2  7  8
      0  0  1
      1  3  4
      2  6  7
```

Change the order of the columns

```
[20]: df = pd.concat([A, B], ignore_index=True)
df
```

```
[20]:      A  B  C   D
0  0.0  1  2  NaN
1  3.0  4  5  NaN
2  6.0  7  8  NaN
3  NaN  0  1  2.0
4  NaN  3  4  5.0
5  NaN  6  7  8.0
```

Rotate columns

```
[21]: df = df.reindex(columns=["h", "C", "B", "A"])
df
```

```
[21]:      h  C  B   A
0  NaN  2  1  0.0
1  NaN  5  4  3.0
2  NaN  8  7  6.0
3  NaN  1  0  NaN
4  NaN  4  3  NaN
5  NaN  7  6  NaN
```

Change the order (the index) of the rows

```
[23]: df.reindex([3,4, 5, 2, 1, 0])
```

```
[23]:      h  C  B   A
3  NaN  1  0  NaN
4  NaN  4  3  NaN
5  NaN  7  6  NaN
2  NaN  8  7  6.0
1  NaN  5  4  3.0
0  NaN  2  1  0.0
```

```
[24]: df.index.values[0] = df.index.values[3]
df.reindex([5])
```

```
[24]:      h  C  B   A
5  NaN  7  6  NaN
```

Pandas Combining Datasets Merge

December 30, 2022

1 Merge and Join

`pd.merge()` lets you efficiently link data from different sources

The behavior implemented in `pd.merge()` is a subset of what is known as relational algebra, which is a formal set of rules for manipulating relational data

The `pd.merge()` function implements a number of types of joins: the one-to-one, many-to-one, and many-to-many joins

One-to-one joins

```
[1]: import pandas as pd
```

one-to-one join is very similar to the column-wise concatenation

```
[2]: # df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue'],
#                           'group': ['Accounting', 'Engineering', 'Engineering',
#                                     'HR']})
df2 = pd.DataFrame({'employee': ['Lisa', 'Sue', 'Bob', 'Jake'],
                    'hire_date': [2004, 2008, 2012, 2014]})
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue', 'Mark'],
                    'group': ['Accounting', 'Engineering', 'Engineering',
                              'HR', 'HR']})
df1
```

```
[2]:   employee  group
0      Bob  Accounting
1      Jake  Engineering
2      Lisa  Engineering
3       Sue           HR
4      Mark           HR
```

```
[3]: df2
```

```
[3]:   employee  hire_date
0      Lisa      2004
1       Sue      2008
2       Bob      2012
3       Jake      2014
```

```
[4]: df3 = pd.merge(df1, df2)
df3
```

```
[4]:   employee      group  hire_date
0      Bob  Accounting      2012
1      Jake Engineering      2014
2      Lisa Engineering      2004
3      Sue           HR       2008
```

The `pd.merge()` function recognizes that each DataFrame has an “employee” column, and automatically joins using this column as a key. the merge in general discards the index.

By default, the result contains the intersection of the two sets of inputs

Many-to-one joins

Many-to-one joins are joins in which one of the two key columns contains duplicate entries

```
[5]: df4 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR'],
                        'supervisor': ['Carly', 'Guido', 'Steve']})
df4
```

```
[5]:      group supervisor
0  Accounting      Carly
1  Engineering      Guido
2           HR      Steve
```

```
[6]: pd.merge(df3, df4)
```

```
[6]:   employee      group  hire_date supervisor
0      Bob  Accounting      2012      Carly
1      Jake Engineering      2014      Guido
2      Lisa Engineering      2004      Guido
3      Sue           HR       2008      Steve
```

Many-to-many joins

If the key column in both the left and right array contains duplicates, then the result is a many-to-many merge.

```
[7]: df5 = pd.DataFrame({'group': ['Accounting', 'Accounting',
                                   'Engineering', 'Engineering', 'HR', 'HR'],
                        'skills': ['math', 'spreadsheets', 'coding', 'linux',
                                   'spreadsheets', 'organization']})
df5
```

```
[7]:      group      skills
0  Accounting      math
1  Accounting spreadsheets
2  Engineering      coding
```

```

3 Engineering      linux
4           HR  spreadsheets
5           HR  organization

```

```

[8]: df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue', 'Mark'],
                        'group': ['Accounting', 'Engineering', 'Engineering',
                                ↪ 'HR', 'HR']})
df1

```

```

[8]:   employee      group
0      Bob  Accounting
1      Jake  Engineering
2      Lisa  Engineering
3       Sue           HR
4      Mark           HR

```

```

[9]: pd.merge(df1, df5)

```

```

[9]:   employee      group      skills
0      Bob  Accounting         math
1      Bob  Accounting  spreadsheets
2      Jake  Engineering        coding
3      Jake  Engineering         linux
4      Lisa  Engineering        coding
5      Lisa  Engineering         linux
6       Sue           HR  spreadsheets
7       Sue           HR  organization
8      Mark           HR  spreadsheets
9      Mark           HR  organization

```

Specification of the Merge Key

The ON keyword

on : label or list Field names to join on. Must be found in both DataFrames.

```

[10]: pd.merge(df1, df2, on='employee')

```

```

[10]:   employee      group  hire_date
0      Bob  Accounting      2012
1      Jake  Engineering      2014
2      Lisa  Engineering      2004
3       Sue           HR      2008

```

The left_on and right_on keywords

At times you may wish to merge two datasets with different column names


```
[11]: df6 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue', 'Alan'],
                        'salary': [70000, 80000, 120000, 90000, 100000]})
df6
```

```
[11]:   name  salary
0   Bob   70000
1   Jake   80000
2   Lisa  120000
3    Sue   90000
4   Alan  100000
```

```
[12]: pd.merge(df1, df6, left_on="employee", right_on="name")
```

```
[12]:   employee      group  name  salary
0      Bob  Accounting   Bob   70000
1      Jake  Engineering  Jake   80000
2      Lisa  Engineering  Lisa  120000
3      Sue           HR    Sue   90000
```

The `left_index` and `right_index` keywords

Sometimes, rather than merging on a column, you would instead like to merge on an index

```
[13]: df1a = df1.set_index('employee')
df2a = df2.set_index('employee')
print(df1a)
df2a
```

```

              group
employee
Bob      Accounting
Jake      Engineering
Lisa      Engineering
Sue              HR
Mark              HR
```

```
[13]:   hire_date
employee
Lisa      2004
Sue       2008
Bob       2012
Jake      2014
```

You can use the index as the key for merging by specifying the `left_index` and/or `right_index` flags in `pd.merge()`

```
[14]: pd.merge(df1a, df2a, left_index=True, right_index=True)
```

```
[14]:
```

	group	hire_date
employee		
Bob	Accounting	2012
Jake	Engineering	2014
Lisa	Engineering	2004
Sue	HR	2008

For convenience, DataFrames implement the `join()` method, which performs a merge that defaults to joining on indices

```
[15]: print(df1a)
df1a.join(df2a)
```

	group
employee	
Bob	Accounting
Jake	Engineering
Lisa	Engineering
Sue	HR
Mark	HR

```
[15]:
```

	group	hire_date
employee		
Bob	Accounting	2012.0
Jake	Engineering	2014.0
Lisa	Engineering	2004.0
Sue	HR	2008.0
Mark	HR	NaN

If you'd like to mix indices and columns, you can combine `left_index` with `right_on` or `left_on` with `right_index`

```
[16]: df1a
```

```
[16]:
```

	group
employee	
Bob	Accounting
Jake	Engineering
Lisa	Engineering
Sue	HR
Mark	HR

```
[17]: df6
```

```
[17]:
```

	name	salary
0	Bob	70000
1	Jake	80000
2	Lisa	120000

```
3 Sue 90000
4 Alan 100000
```

```
[18]: pd.merge(df1a, df6, left_index=True, right_on='name')
```

```
[18]:      group  name  salary
0  Accounting  Bob   70000
1  Engineering  Jake   80000
2  Engineering  Lisa  120000
3           HR   Sue   90000
```

Specifying Set Arithmetic for Joins

```
[19]: df6 = pd.DataFrame({'name': ['Peter', 'Paul', 'Mary'],
                        'food': ['fish', 'beans', 'bread']},
                        columns=['name', 'food'])
df7 = pd.DataFrame({'name': ['Mary', 'Joseph'],
                    'drink': ['wine', 'beer']},
                    columns=['name', 'drink'])

print(df6)
df7
```

```
      name  food
0  Peter  fish
1   Paul  beans
2   Mary  bread
```

```
[19]:      name drink
0   Mary  wine
1  Joseph  beer
```

inner join

```
[20]: pd.merge(df6, df7)
```

```
[20]:      name  food drink
0   Mary  bread  wine
```

outer join

```
[21]: pd.merge(df6, df7, how='outer')
```

```
[21]:      name  food drink
0   Peter  fish   NaN
1   Paul  beans   NaN
2   Mary  bread  wine
3  Joseph   NaN  beer
```

```
[22]: df6
```

```
[22]:      name  food
0  Peter  fish
1   Paul  beans
2   Mary  bread
```

```
[23]: df7
```

```
[23]:      name drink
0   Mary  wine
1  Joseph  beer
```

left and right

```
[24]: print(df6)
pd.merge(df6, df7, how='left')
```

```
      name  food
0  Peter  fish
1   Paul  beans
2   Mary  bread
```

```
[24]:      name  food drink
0  Peter  fish   NaN
1   Paul  beans   NaN
2   Mary  bread  wine
```

```
[25]: pd.merge(df6, df7, how='right')
```

```
[25]:      name  food drink
0   Mary  bread  wine
1  Joseph   NaN  beer
```

Overlapping Column Names: The suffixes Keyword

You may end up in a case where your two input DataFrames have conflicting column names

```
[26]: df8 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],
                          'rank': [1, 2, 3, 4]})
df9 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],
                     'rank': [3, 1, 4, 2]})
df8
```

```
[26]:      name  rank
0   Bob      1
1  Jake      2
2  Lisa      3
```

3 Sue 4

```
[27]: pd.merge(df8, df9, on="name")
```

```
[27]:
```

	name	rank_x	rank_y
0	Bob	1	3
1	Jake	2	1
2	Lisa	3	4
3	Sue	4	2

Because the output would have two conflicting column names, the merge function automatically appends a suffix `_x` or `_y` to make the output columns unique

```
[28]: pd.merge(df8, df9, on="name", suffixes=["_L", "_R"])
```

```
[28]:
```

	name	rank_L	rank_R
0	Bob	1	3
1	Jake	2	1
2	Lisa	3	4
3	Sue	4	2

Pandas Groupby

December 30, 2022

1 Groupby method

```
[1]: import pandas as pd
```

```
[2]: data = pd.read_csv('phone_data.csv')
data.head()
```

```
[2]:
```

	index	date	duration	item	month	network	network_type
0	0	15/10/14 06:58	34.429	data	2014-11	data	data
1	1	15/10/14 06:58	13.000	call	2014-11	Vodafone	mobile
2	2	15/10/14 14:46	23.000	call	2014-11	Meteor	mobile
3	3	15/10/14 14:48	4.000	call	2014-11	Tesco	mobile
4	4	15/10/14 17:27	4.000	call	2014-11	Tesco	mobile

Groupby essentially splits the data into different groups depending on a variable of your choice. For example, the expression `data.groupby('month')` will split our current DataFrame by month.

The `groupby()` function returns a GroupBy object, but essentially describes how the rows of the original data set has been split. the GroupBy object `.groups` variable is a dictionary whose keys are the computed unique groups and corresponding values being the axis labels belonging to each group.

```
[3]: data.groupby(['month'])
```

```
[3]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x7f80083c9220>
```

```
[5]: data.groupby(['month']).groups.keys()
```

```
[5]: dict_keys(['2014-11', '2014-12', '2015-01', '2015-02', '2015-03'])
```

```
[6]: len(data.groupby(['month']).groups['2014-11'])
```

```
[6]: 230
```

Functions like `max()`, `min()`, `mean()`, `first()`, `last()` can be quickly applied to the GroupBy object to obtain summary statistics for each group – an immensely useful function.

`first()` method for subsetting initial periods of time series data

```
[9]: data.groupby('month').first()
```

```
[9]:
```

	index	date	duration	item	network	network_type
month						
2014-11	0	15/10/14 06:58	34.429	data	data	data
2014-12	228	13/11/14 06:58	34.429	data	data	data
2015-01	381	13/12/14 06:58	34.429	data	data	data
2015-02	577	13/01/15 06:58	34.429	data	data	data
2015-03	729	12/02/15 20:15	69.000	call	landline	landline

Get the sum of the durations per month

```
[10]: data.groupby('month')['duration'].sum()
```

```
[10]: month
2014-11    26639.441
2014-12    14641.870
2015-01    18223.299
2015-02    15522.299
2015-03    22750.441
Name: duration, dtype: float64
```

Get the number of dates / entries in each month

```
[11]: data.groupby('month')['date'].count()
```

```
[11]: month
2014-11     230
2014-12     157
2015-01     205
2015-02     137
2015-03     101
Name: date, dtype: int64
```

What is the sum of durations, for calls only, to each network

```
[12]: data[data['item'] == 'call'].groupby('network')['duration'].sum()
```

```
[12]: network
Meteor      7200.0
Tesco      13828.0
Three     36464.0
Vodafone   14621.0
landline   18433.0
voicemail   1775.0
Name: duration, dtype: float64
```

How many calls, sms, and data entries are in each month?

```
[13]: data.groupby(['month', 'item'])['date'].count()
```

```
[13]: month    item
      2014-11  call    107
              data     29
              sms     94
      2014-12  call     79
              data     30
              sms     48
      2015-01  call     88
              data     31
              sms     86
      2015-02  call     67
              data     31
              sms     39
      2015-03  call     47
              data     29
              sms     25
      Name: date, dtype: int64
```

How many calls, texts, and data are sent per month, split by network_type?

```
[14]: data.groupby(['month', 'network_type'])['date'].count()
```

```
[14]: month    network_type
      2014-11  data           29
              landline        5
              mobile        189
              special         1
              voicemail        6
      2014-12  data           30
              landline         7
              mobile        108
              voicemail         8
              world           4
      2015-01  data           31
              landline        11
              mobile        160
              voicemail         3
      2015-02  data           31
              landline         8
              mobile         90
              special          2
              voicemail         6
      2015-03  data           29
              landline        11
              mobile         54
```


voicemail	4
world	3

Name: date, dtype: int64

Visualization Matplotlib

December 30, 2022

1 Data Visualization with Matplotlib

Matplotlib is a multi-platform data visualization library built on NumPy arrays. Matplotlib supports dozens of backends and output types. The cross-platform, everything-to-everyone approach has been one of the great strengths of Matplotlib.

```
[9]: import matplotlib.pyplot as plt
```

We will use the `plt.style` directive to choose appropriate aesthetic styles for our figures. Here we will set the classic style, which ensures that the plots we create use the classic Matplotlib style:

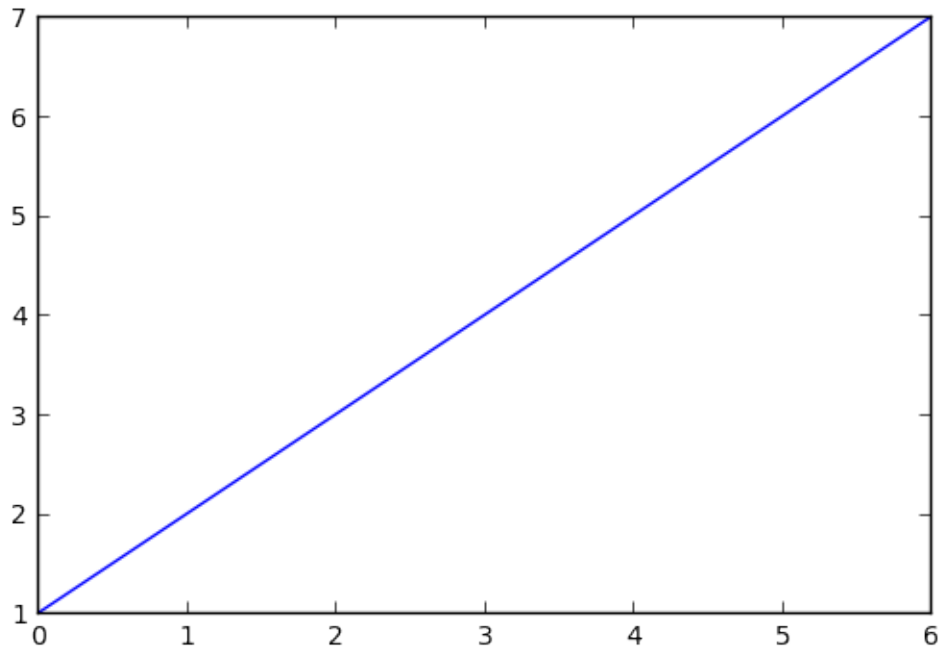
```
[10]: # plt.style.use('classic')
```

How to Display Your Plots

`plt.show()` starts an event loop, looks for all currently active figure objects, and opens one or more interactive windows that display your figure or figures.

```
[11]: data = [1,2,3,4,5,6,7]
```

```
[12]: plt.plot(data)
      plt.show()
```



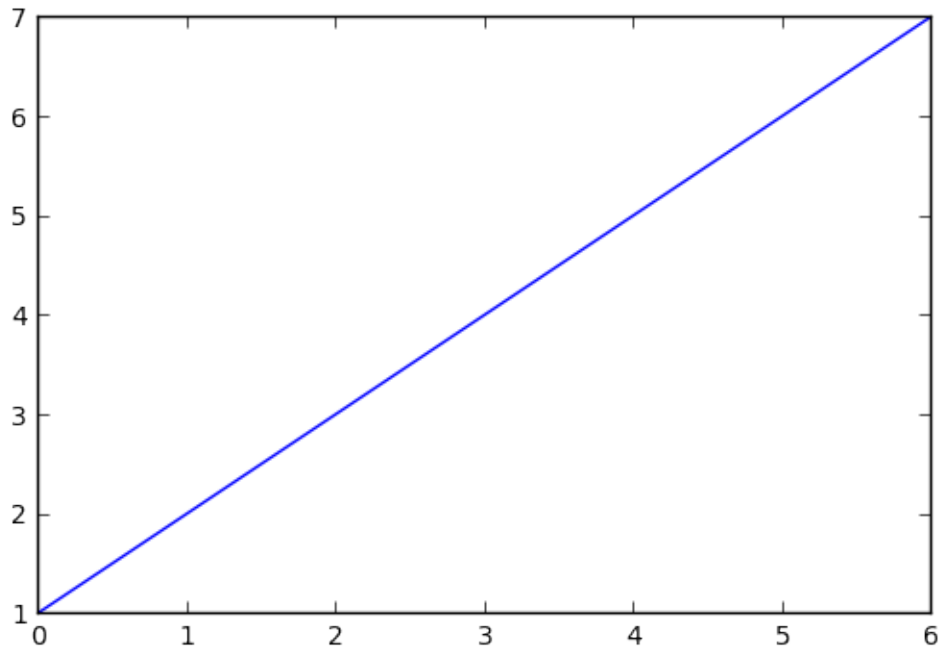
The `plt.show()` command does a lot under the hood, as it must interact with your system's interactive graphical backend.

the `plt.show()` command should be used only once per Python session, and is most often seen at the very end of the script. Multiple `show()` commands can lead to unpredictable backend-dependent behavior, and should mostly be avoided.

Saving Figures to File

Matplotlib has the ability to save figures in a wide variety of formats

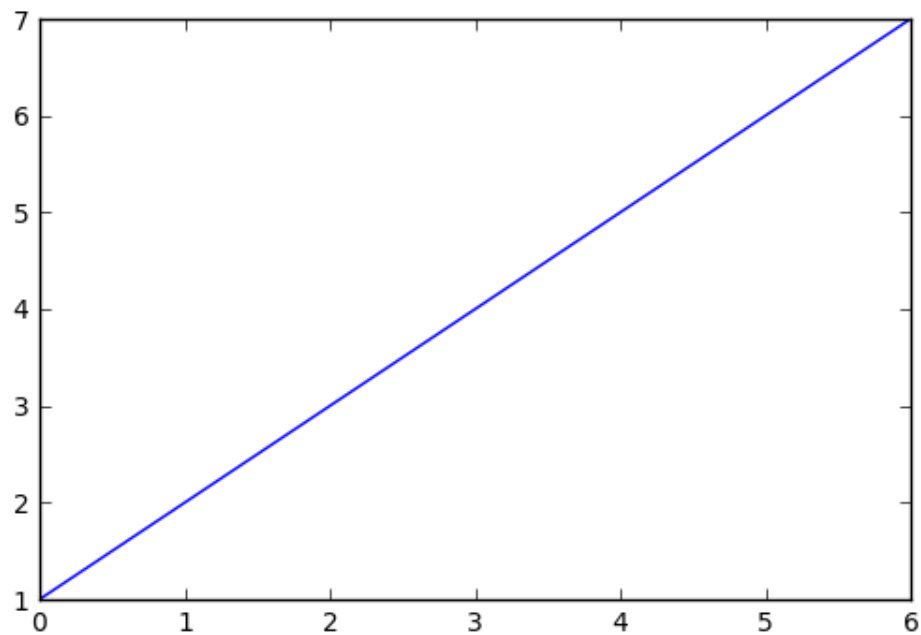
```
[14]: plt.plot(data)
      plt.savefig('my_figure.png')
```



To confirm that my_figure.png contains img use the IPython Image object to display the contents of the file.

```
[15]: from IPython.display import Image  
      Image('my_figure.png')
```

[15]:



savefig() supports many different file formats

```
'eps': 'Encapsulated Postscript',  
'jpeg': 'Joint Photographic Experts Group',  
'jpg': 'Joint Photographic Experts Group',  
'pdf': 'Portable Document Format',  
'pgf': 'PGF code for LaTeX',  
'png': 'Portable Network Graphics',  
'ps': 'Postscript',  
'raw': 'Raw RGBA bitmap',  
'rgba': 'Raw RGBA bitmap',  
'svg': 'Scalable Vector Graphics',  
'svgz': 'Scalable Vector Graphics',  
'tif': 'Tagged Image File Format',  
'tiff': 'Tagged Image File Format'
```

[]:

Matplotlib Line Plots

December 30, 2022

1 Line Plots

The simplest of all plots is the visualization of a single function $y=f(x)$

```
[5]: import numpy as np
      %matplotlib inline
      import matplotlib.pyplot as plt
```

```
[6]: # plt.style.use('grayscale')
```

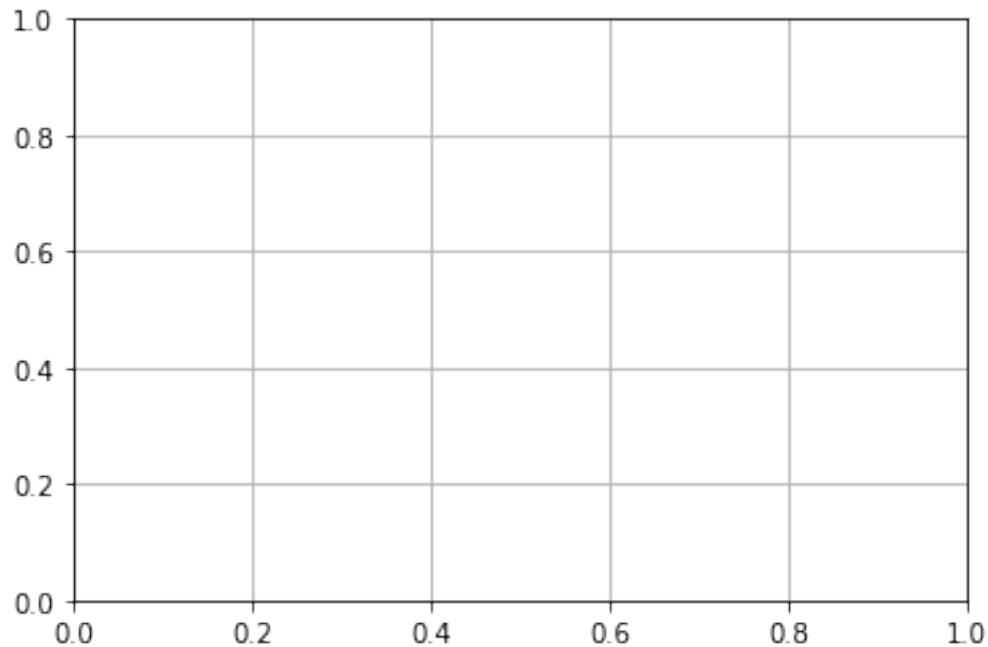
For all Matplotlib plots, we start by creating a figure and an axes.

In Matplotlib, the figure (an instance of the class `plt.Figure`) can be thought of as a single container that contains all the objects representing axes, graphics, text, and labels.

The axes (an instance of the class `plt.Axes`) is what we see above: a bounding box with ticks and labels, which will eventually contain the plot elements that make up our visualization.

```
[7]: #Object-oriented interface

fig = plt.figure()
ax = plt.axes()
# x = np.linspace(0, 10, 1000)
# ax.plot(x, np.sin(x))
plt.grid()
```



Once we have created an axes, we can use the `ax.plot` function to plot some data.

```
[8]: x = [1,2,3,5,5,6,7]
     y = [1,6,3,4,5,6,7]
```

`plot()` - Plot lines and/or markers to the Axes. `plot()` is a versatile command, and will take an arbitrary number of arguments. For example, to plot `x` versus `y`, you can issue the command:

```
[15]: help(plt.plot)
```

Help on function `plot` in module `matplotlib.pyplot`:

```
plot(*args, **kwargs)
```

Plot lines and/or markers to the

`:class:~matplotlib.axes.Axes`.` `*args*` is a variable length argument, allowing for multiple `*x*`, `*y*` pairs with an optional format string. For example, each of the following is legal::

```
plot(x, y)           # plot x and y using default line style and color
plot(x, y, 'bo')      # plot x and y using blue circle markers
plot(y)              # plot y using x as index array 0..N-1
plot(y, 'r+')         # ditto, but with red plusses
```

If `*x*` and/or `*y*` is 2-dimensional, then the corresponding columns will be plotted.

If used with labeled data, make sure that the color spec is not included as an element in data, as otherwise the last case
```plot("v","r", data={"v":..., "r":...})```  
 can be interpreted as the first case which would do ```plot(v, r)```  
 using the default line style and color.

If not used with labeled data (i.e., without a data argument),  
 an arbitrary number of `*x*`, `*y*`, `*fmt*` groups can be specified, as in::

```
a.plot(x1, y1, 'g^', x2, y2, 'g-')
```

Return value is a list of lines that were added.

By default, each line is assigned a different style specified by a  
 'style cycle'. To change this behavior, you can edit the  
`axes.prop_cycle` rcParam.

The following format string characters are accepted to control  
 the line style or marker:

character	description
``_``	solid line style
``_``	dashed line style
``_``	dash-dot line style
``:``	dotted line style
``.``	point marker
``,'``	pixel marker
``o``	circle marker
``v``	triangle_down marker
``^``	triangle_up marker
``<``	triangle_left marker
``>``	triangle_right marker
``1``	tri_down marker
``2``	tri_up marker
``3``	tri_left marker
``4``	tri_right marker
``s``	square marker
``p``	pentagon marker
``*``	star marker
``h``	hexagon1 marker
``H``	hexagon2 marker
``+``	plus marker
``x``	x marker
``D``	diamond marker
``d``	thin_diamond marker



```

'|' vline marker
'|_' hline marker
=====

```

The following color abbreviations are supported:

```

=====
character color
=====
'b' blue
'g' green
'r' red
'c' cyan
'm' magenta
'y' yellow
'k' black
'w' white
=====

```

In addition, you can specify colors in many weird and wonderful ways, including full names (`'green'`), hex strings (`'#008000'`), RGB or RGBA tuples (`(0,1,0,1)`) or grayscale intensities as a string (`'0.8'`). Of these, the string specifications can be used in place of a `'fmt'` group, but the tuple forms can be used only as `'kwargs'`.

Line styles and colors are combined in a single format string, as in `'bo'` for blue circles.

The `*kwargs` can be used to set line properties (any property that has a `'set_'` method). You can use this to set a line label (for auto legends), linewidth, antialiasing, marker face color, etc. Here is an example::

```

plot([1,2,3], [1,2,3], 'go-', label='line 1', linewidth=2)
plot([1,2,3], [1,4,9], 'rs', label='line 2')
axis([0, 4, 0, 10])
legend()

```

If you make multiple lines with one plot command, the `kwargs` apply to all those lines, e.g.::

```

plot(x1, y1, x2, y2, antialiased=False)

```

Neither line will be antialiased.

You do not need to use format strings, which are just

abbreviations. All of the line properties can be controlled by keyword arguments. For example, you can set the color, marker, linestyle, and markercolor with::

```
plot(x, y, color='green', linestyle='dashed', marker='o',
 markerfacecolor='blue', markersize=12).
```

See :class:`~matplotlib.lines.Line2D` for details.

The kwargs are :class:`~matplotlib.lines.Line2D` properties:

```
agg_filter: a filter function, which takes a (m, n, 3) float array and a
dpi value, and returns a (m, n, 3) array
alpha: float (0.0 transparent through 1.0 opaque)
animated: bool
antialiased or aa: [True | False]
clip_box: a `~.Bbox` instance
clip_on: bool
clip_path: [(~matplotlib.path.Path`, ~.Transform`) | ~.Patch` | None]
color or c: any matplotlib color
contains: a callable function
dash_capstyle: ['butt' | 'round' | 'projecting']
dash_joinstyle: ['miter' | 'round' | 'bevel']
dashes: sequence of on/off ink in points
drawstyle: ['default' | 'steps' | 'steps-pre' | 'steps-mid' | 'steps-
post']
figure: a ~.Figure` instance
fillstyle: ['full' | 'left' | 'right' | 'bottom' | 'top' | 'none']
gid: an id string
label: object
linestyle or ls: ['solid' | 'dashed', 'dashdot', 'dotted' | (offset, on-
off-dash-seq) | '--' | '-.-' | '-.' | ':' | 'None' | ' ' |
' '']
linewidth or lw: float value in points
marker: :mod:`A valid marker style <matplotlib.markers>`
markeredgecolor or mec: any matplotlib color
markeredgewidth or mew: float value in points
markerfacecolor or mfc: any matplotlib color
markerfacecoloralt or mfcalt: any matplotlib color
markersize or ms: float
markevery: [None | int | length-2 tuple of int | slice | list/array of int
| float | length-2 tuple of float]
path_effects: ~.AbstractPathEffect`
picker: float distance in points or callable pick function ``fn(artist,
event)``
pickradius: float distance in points
rasterized: bool or None
sketch_params: (scale: float, length: float, randomness: float)
```

snap: bool or None  
solid\_capstyle: ['butt' | 'round' | 'projecting']  
solid\_joinstyle: ['miter' | 'round' | 'bevel']  
transform: a :class:`matplotlib.transforms.Transform` instance  
url: a url string  
visible: bool  
xdata: 1D array  
ydata: 1D array  
zorder: float

kwargs *\*scalex\** and *\*scaley\**, if defined, are passed on to  
:meth:`~matplotlib.axes.Axes.autoscale\_view` to determine  
whether the *\*x\** and *\*y\** axes are autoscaled; the default is  
*\*True\**.

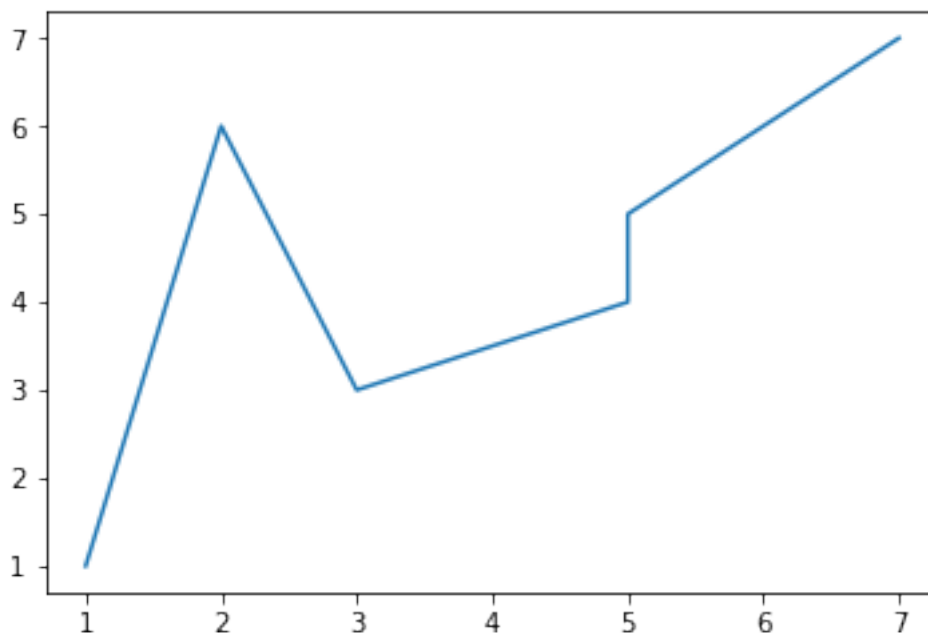
.. note::

In addition to the above described arguments, this function can take a  
*\*\*data\*\** keyword argument. If such a *\*\*data\*\** argument is given, the  
following arguments are replaced by *\*\*data[<arg>]\*\**:

\* All arguments with the following names: 'x', 'y'.

```
[14]: #MATLAB-style Interface
plt.plot(x,y)
```

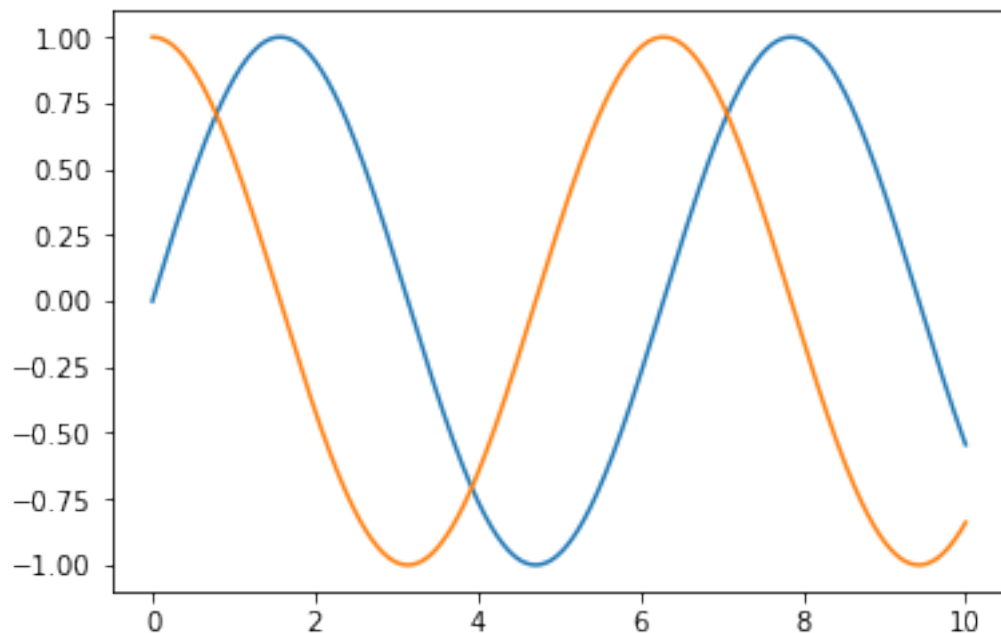
```
[14]: [<matplotlib.lines.Line2D at 0x10c1ecf28>]
```



For every x, y pair of arguments, there is an optional third argument which is the format string that indicates the color and line type of the plot. The letters and symbols of the format string are from MATLAB, and you concatenate a color string with a line style string.

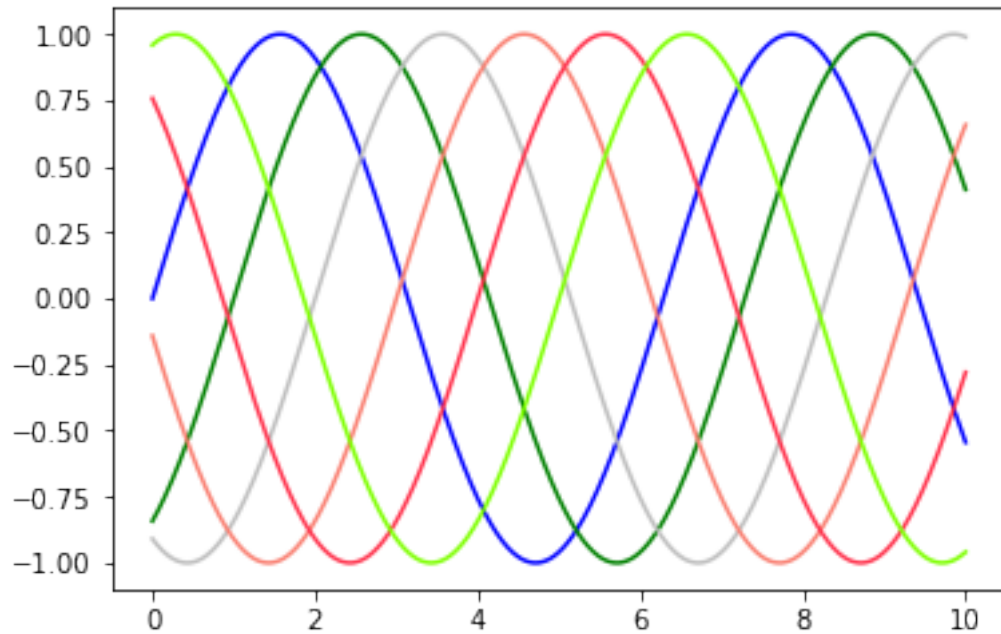
```
[11]: x = np.linspace(0, 10, 1000)
 # print(x)
 plt.plot(x, np.sin(x))
 plt.plot(x, np.cos(x))
```

```
[11]: [<matplotlib.lines.Line2D at 0x114832e80>]
```



### Line Colors and Styles

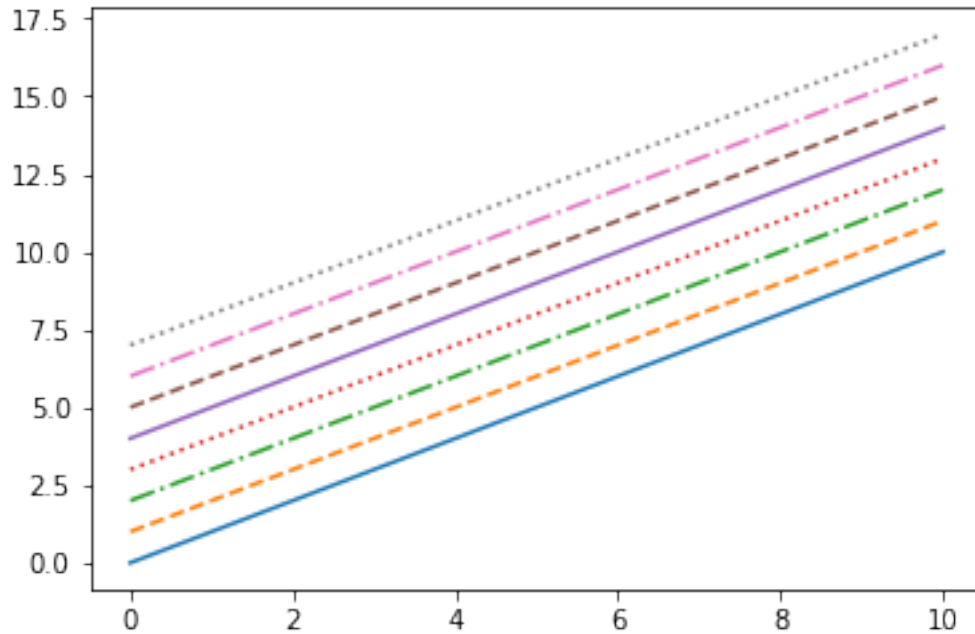
```
[14]: plt.plot(x, np.sin(x - 0), color='blue') # specify color by name
 plt.plot(x, np.sin(x - 1), color='g') # short color code (rgbcmyk)
 plt.plot(x, np.sin(x - 2), color='0.75') # Grayscale between 0 and 1
 plt.plot(x, np.sin(x - 3), color='#FA8072') # Hex code (RRGGBB from 00 to FF)
 plt.plot(x, np.sin(x - 4), color=(1.0,0.2,0.3)) # RGB tuple, values 0 to 1
 plt.plot(x, np.sin(x - 5), color='chartreuse'); # all HTML color names supported
```



Similarly, the line style can be adjusted using the linestyle keyword

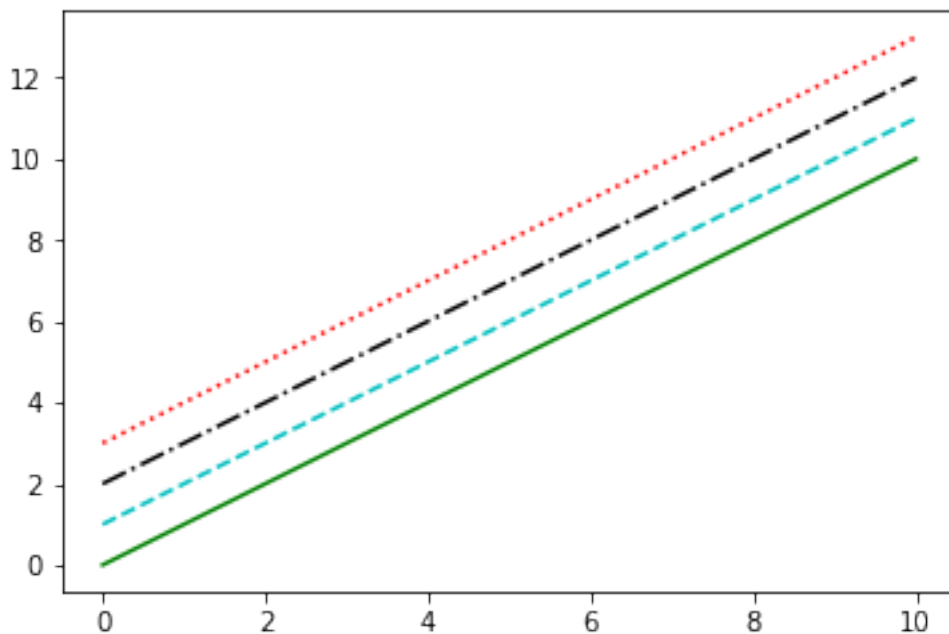
```
[13]: plt.plot(x, x + 0, linestyle='solid')
plt.plot(x, x + 1, linestyle='dashed')
plt.plot(x, x + 2, linestyle='dashdot')
plt.plot(x, x + 3, linestyle='dotted');

For short, you can use the following codes:
plt.plot(x, x + 4, linestyle='-') # solid
plt.plot(x, x + 5, linestyle='--') # dashed
plt.plot(x, x + 6, linestyle='-.') # dashdot
plt.plot(x, x + 7, linestyle=':'); # dotted
```



These linestyle and color codes can be combined into a single non-keyword argument to the `plt.plot()` function

```
[14]: plt.plot(x, x + 0, '-g') # solid green
 plt.plot(x, x + 1, '--c') # dashed cyan
 plt.plot(x, x + 2, '-.k') # dashdot black
 plt.plot(x, x + 3, ':r'); # dotted red
```



These single-character color codes reflect the standard abbreviations in the RGB (Red/Green/Blue) and CMYK (Cyan/Magenta/Yellow/black) color systems, commonly used for digital color graphics.

### Adjusting the Plot: Axes Limits

x limits and y limits

The most basic way to adjust axis limits is to use the `plt.xlim()` and `plt.ylim()` methods

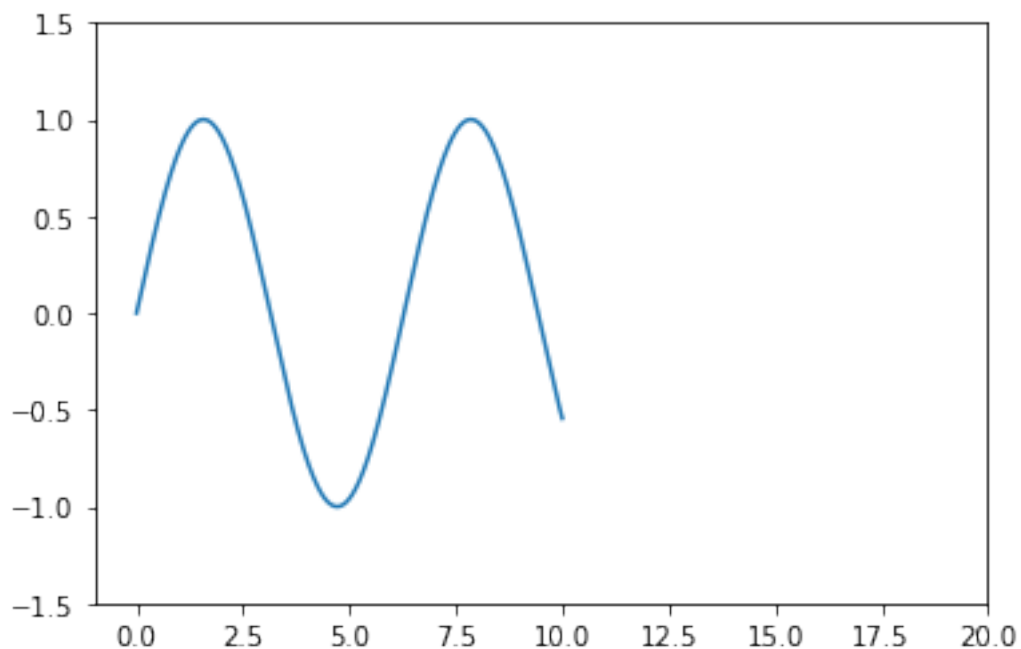
```
[25]: x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x))

adjusting x any limits
plt.xlim(-1, 20)
plt.ylim(-1.5, 1.5)
help(plt.xlim)
plt.xlim(-2, 11)
plt.ylim(-2.5, 1.5)

plt.xlim(0, 10)
plt.grid()

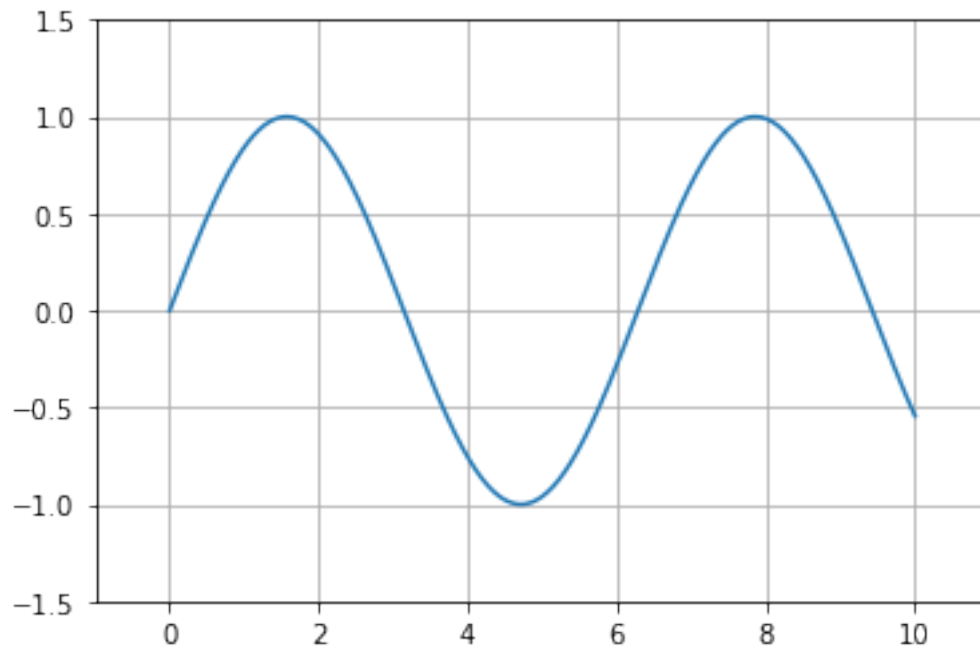
reverse
plt.ylim(1.2, -1.2)
```

[25]: (-1.5, 1.5)



A useful related method is `plt.axis()`. The `plt.axis()` method allows you to set the x and y limits with a single call, by passing a list which specifies `[xmin, xmax, ymin, ymax]`

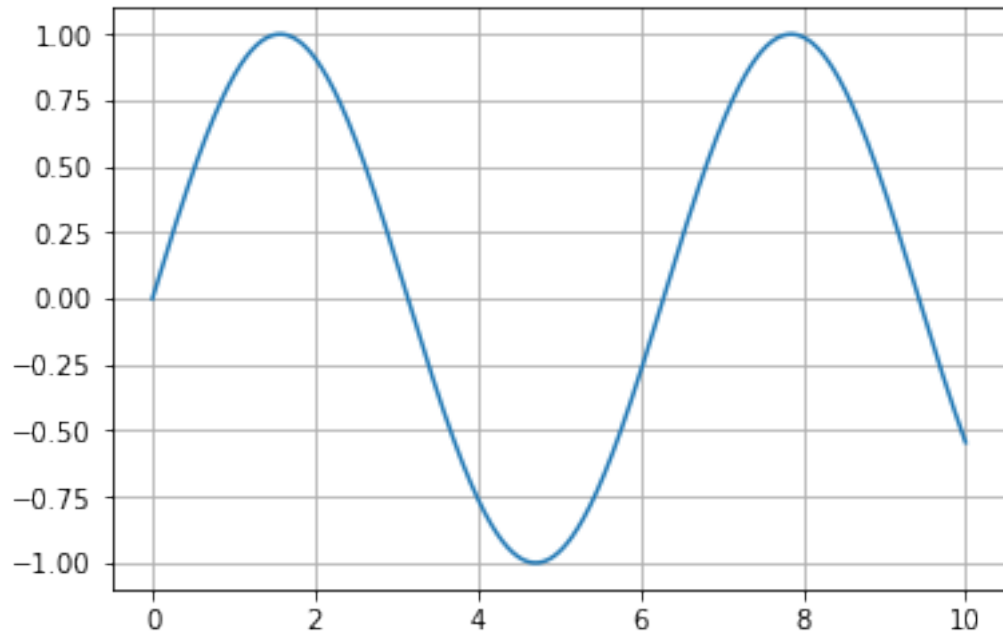
```
[18]: plt.plot(x, np.sin(x))
plt.axis([-1, 11, -1.5, 1.5])
plt.grid()
```



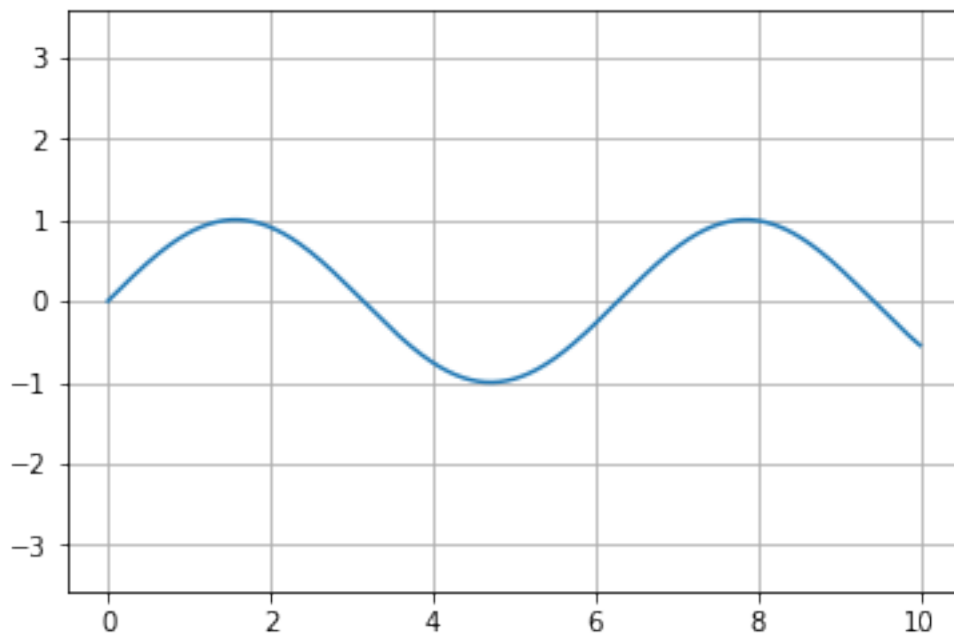
Or you can specify it

```
[19]: plt.plot(x, np.sin(x))
plt.axis('tight')
plt.grid()
```





```
[20]: plt.plot(x, np.sin(x))
 plt.axis('equal')
 plt.grid()
```

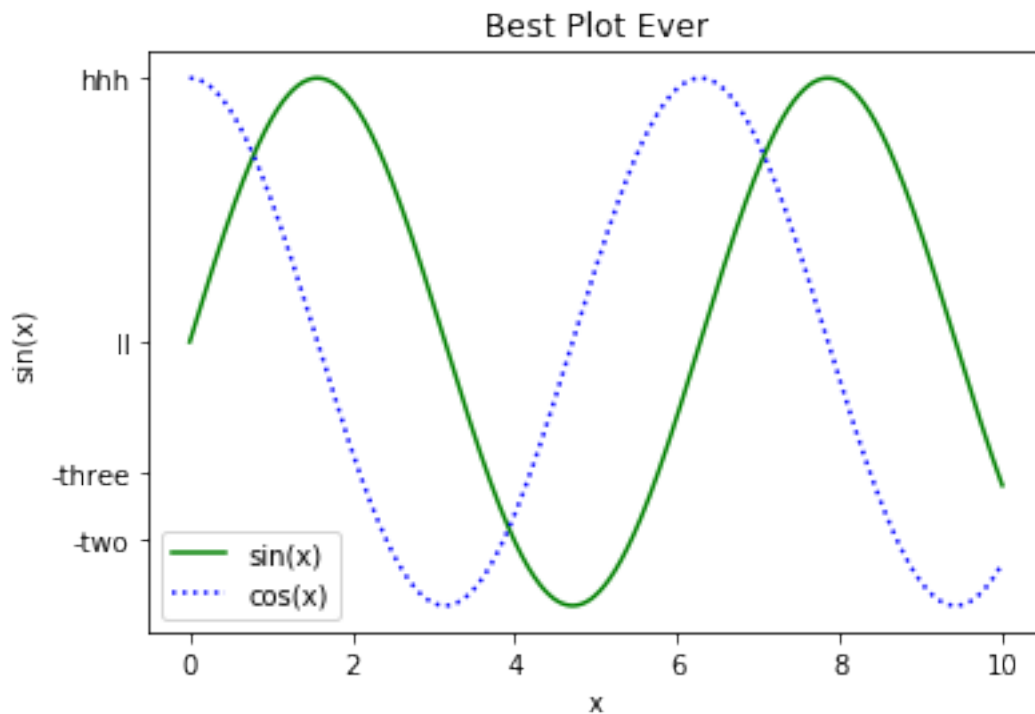


```
[22]: # plt.
```

### Labeling Plots

```
[16]: plt.plot(x, np.sin(x), '-g', label='sin(x)')
plt.plot(x, np.cos(x), ':b', label='cos(x)')
plt.legend()
plt.title("Best Plot Ever")
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.yticks([-1,0,1,2,3,4,5,6,7,8,9,10,11])
you can add labels to Y ticks
plt.yticks([+1.00, -0.75, -0.50,0],['hhh', '-two', '-three','11'])
```

```
[16]: ([<matplotlib.axis.YTick at 0x11cd53d68>,
<matplotlib.axis.YTick at 0x11cd53978>,
<matplotlib.axis.YTick at 0x11cdcb518>,
<matplotlib.axis.YTick at 0x11cbd20b8>],
<a list of 4 Text yticklabel objects>)
```



The position, size, and style of these labels can be adjusted using optional arguments to the function.

```
[]: # help(plt.xlabel)
help(ax.text)
```

```
[24]: from matplotlib.lines import Line2D
 from matplotlib.patches import Rectangle

 # build a rectangle in axes coords
 left, width = .25, .5
 bottom, height = .25, .5
 right = left + width
 top = bottom + height
 ax = plt.gca()
 p = plt.Rectangle((left, bottom), width, height,
 fill=False,
)
 p.set_transform(ax.transAxes)
 p.set_clip_on(False)
 ax.add_patch(p)

 ax.text(left, bottom, 'left top',
 horizontalalignment='left',
 verticalalignment='top',
 transform=ax.transAxes)

 ax.text(left, bottom, 'left bottom',
 horizontalalignment='left',
 verticalalignment='bottom',
 transform=ax.transAxes)

 ax.text(right, top, 'right bottom',
 horizontalalignment='right',
 verticalalignment='bottom',
 transform=ax.transAxes)

 ax.text(right, top, 'right top',
 horizontalalignment='right',
 verticalalignment='top',
 transform=ax.transAxes)

 ax.text(right, bottom, 'center top',
 horizontalalignment='center',
 verticalalignment='top',
 transform=ax.transAxes)

 ax.text(left, 0.5*(bottom + top), 'right center',
 horizontalalignment='right',
 verticalalignment='center',
 rotation='vertical',
 transform=ax.transAxes)
```

```

ax.text(left, 0.5*(bottom + top), 'left center',
 horizontalalignment='left',
 verticalalignment='center',
 rotation='vertical',
 transform=ax.transAxes)

ax.text(0.5*(left + right), 0.5*(bottom + top), 'middle',
 horizontalalignment='center',
 verticalalignment='center',
 transform=ax.transAxes)

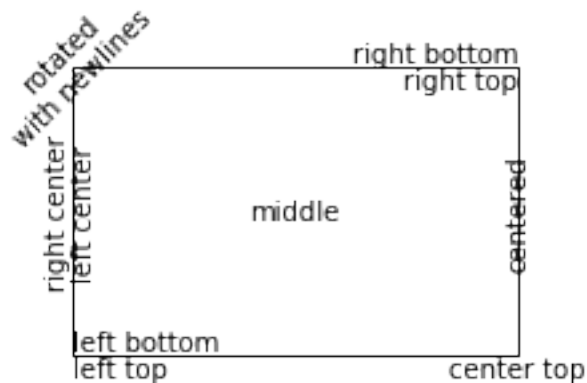
ax.text(right, 0.5*(bottom + top), 'centered',
 horizontalalignment='center',
 verticalalignment='center',
 rotation='vertical',
 transform=ax.transAxes)

ax.text(left, top, 'rotated\nwith newlines',
 horizontalalignment='center',
 verticalalignment='center',
 rotation=45,
 transform=ax.transAxes)

plt.axis('off')

```

[24]: (0.0, 1.0, 0.0, 1.0)

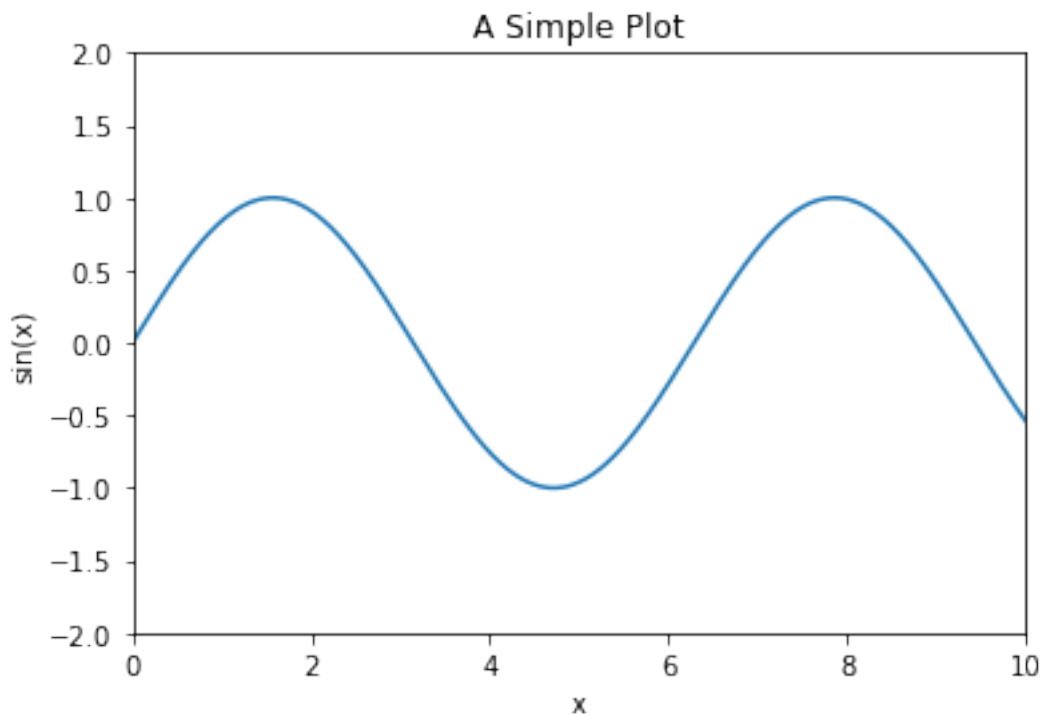


While most plt functions translate directly to ax methods (such as plt.plot() → ax.plot(), plt.legend() → ax.legend(), etc.), this is not the case for all commands. In particular, functions to set limits, labels, and titles are slightly modified.

```
plt.xlabel() → ax.set_xlabel()
plt.ylabel() → ax.set_ylabel()
plt.xlim() → ax.set_xlim()
plt.ylim() → ax.set_ylim()
plt.title() → ax.set_title()
```

```
[25]: #Object-oriented interface
ax = plt.axes()
ax.plot(x, np.sin(x))
ax.set(xlim=(0, 10), ylim=(-2, 2),
 xlabel='x', ylabel='sin(x)',
 title='A Simple Plot')
```

```
[25]: [(-2, 2),
Text(0,0.5,'sin(x)'),
(0, 10),
Text(0.5,0,'x'),
Text(0.5,1,'A Simple Plot')]
```



[ ]:

# Matplotlib Histogram

December 30, 2022

## 1 Histograms

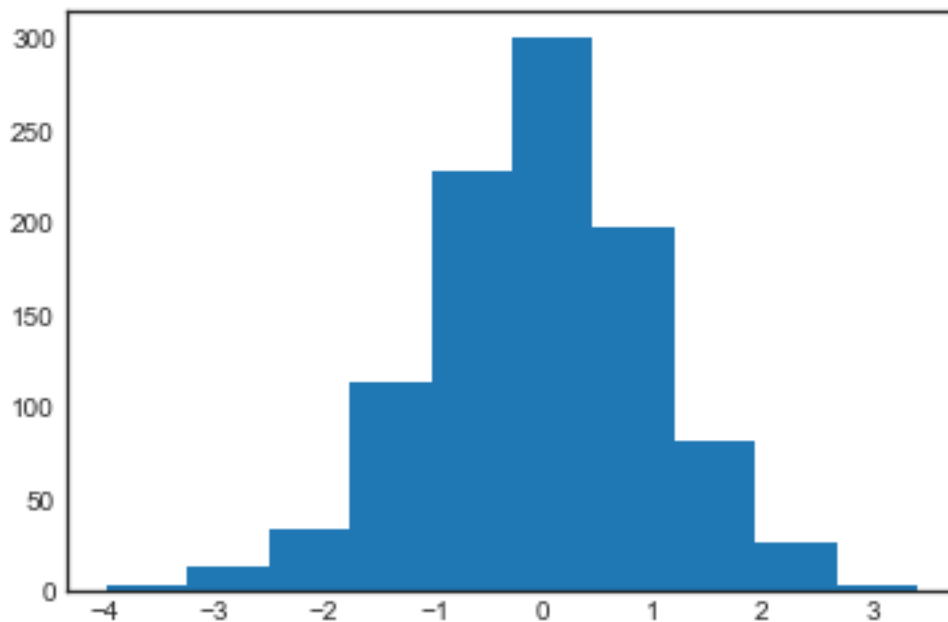
```
[1]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-white')
```

Styles guide

[https://matplotlib.org/devdocs/gallery/style\\_sheets/style\\_sheets\\_reference.html](https://matplotlib.org/devdocs/gallery/style_sheets/style_sheets_reference.html)

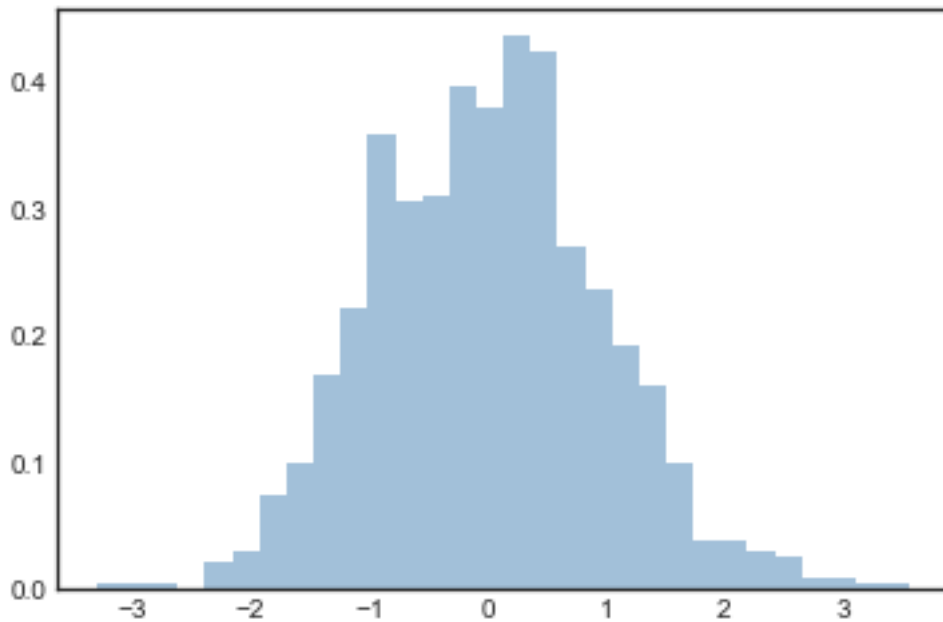
```
[2]: data = np.random.randn(1000)
```

```
[3]: plt.hist(data);
```



```
[8]: # help(plt.hist)
```

```
[15]: plt.hist(data, bins=30, normed=True, alpha=0.5,
 histtype='stepfilled', color='steelblue',
 edgecolor='none')
```



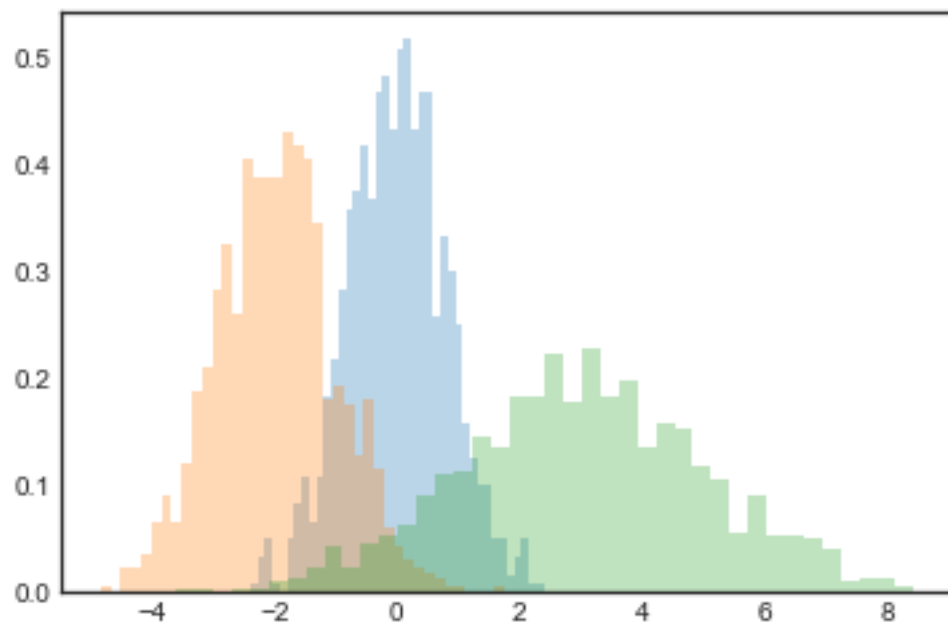
combination of `histtype='stepfilled'` along with some transparency `alpha` to be very useful when comparing histograms of several distributions

```
[16]: x1 = np.random.normal(0, 0.8, 1000)
 x2 = np.random.normal(-2, 1, 1000)
 x3 = np.random.normal(3, 2, 1000)

 kwargs = dict(histtype='stepfilled', alpha=0.3, normed=True, bins=40)

 plt.hist(x1, **kwargs)
 plt.hist(x2, **kwargs)
 plt.hist(x3, **kwargs)
```





[ ]:

# Matplotlib Scatter Plots

December 30, 2022

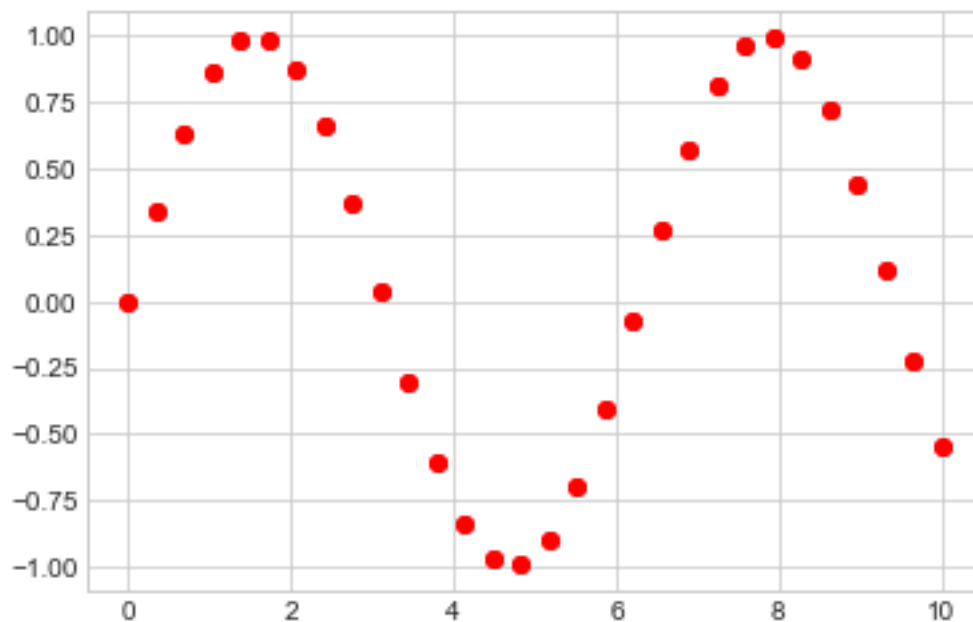
## 1 Scatter Plots

Scatter plot is very close to the line plot

```
[9]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
```

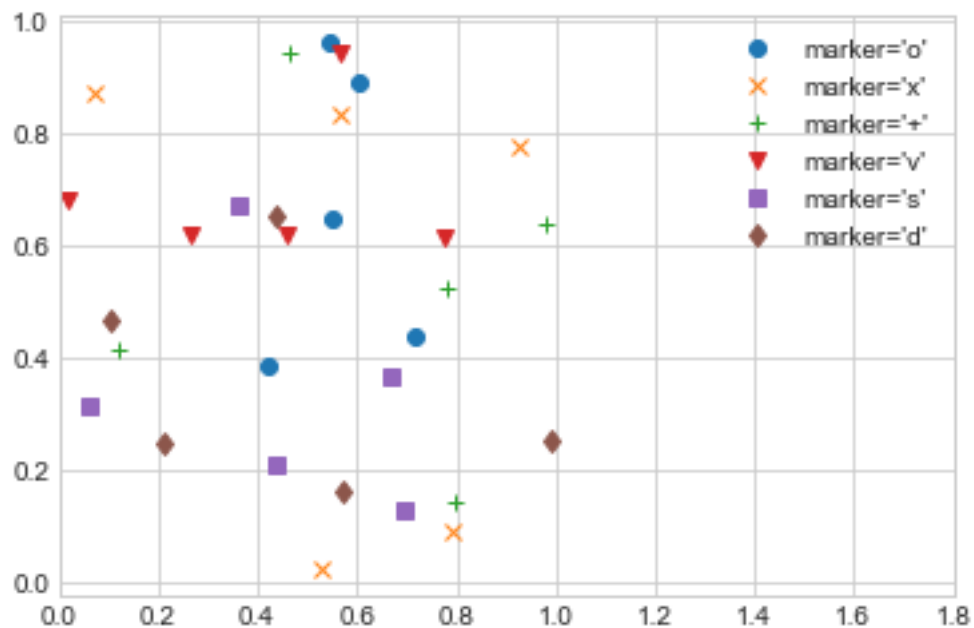
```
[4]: x = np.linspace(0, 10, 30)
y = np.sin(x)

plt.plot(x, y, 'o', color='red')
plt.show()
```



The third argument in the function call is a character that represents the type of symbol used for the plotting.

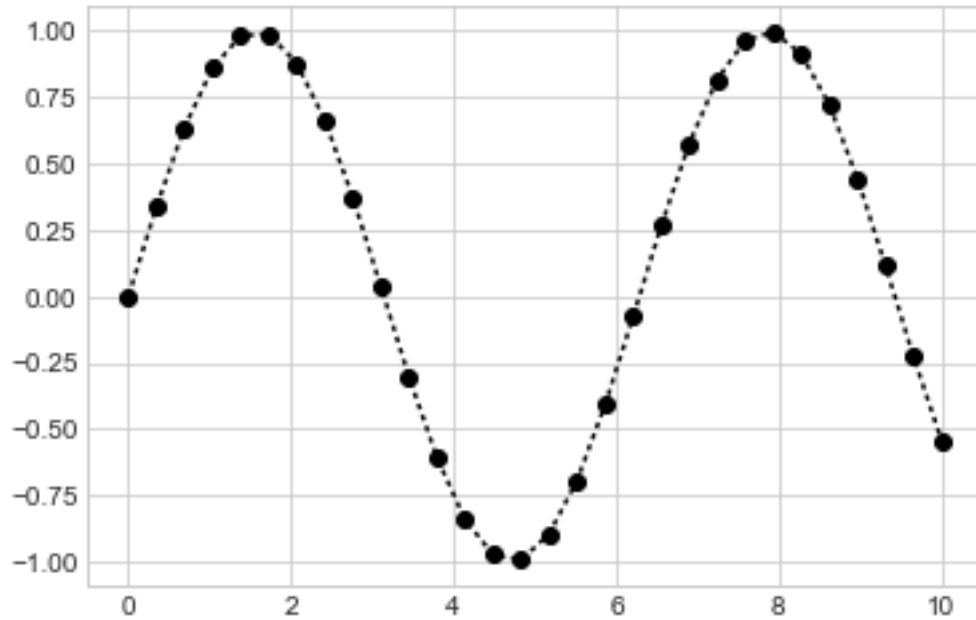
```
[7]: rng = np.random.RandomState(0)
for marker in ['o', 'x', '+', 'v', 's', 'd']:
 plt.plot(rng.rand(5), rng.rand(5), marker,
 label="marker='{0}'".format(marker))
plt.legend(numpoints=1)
plt.xlim(0, 1.8)
plt.show()
```



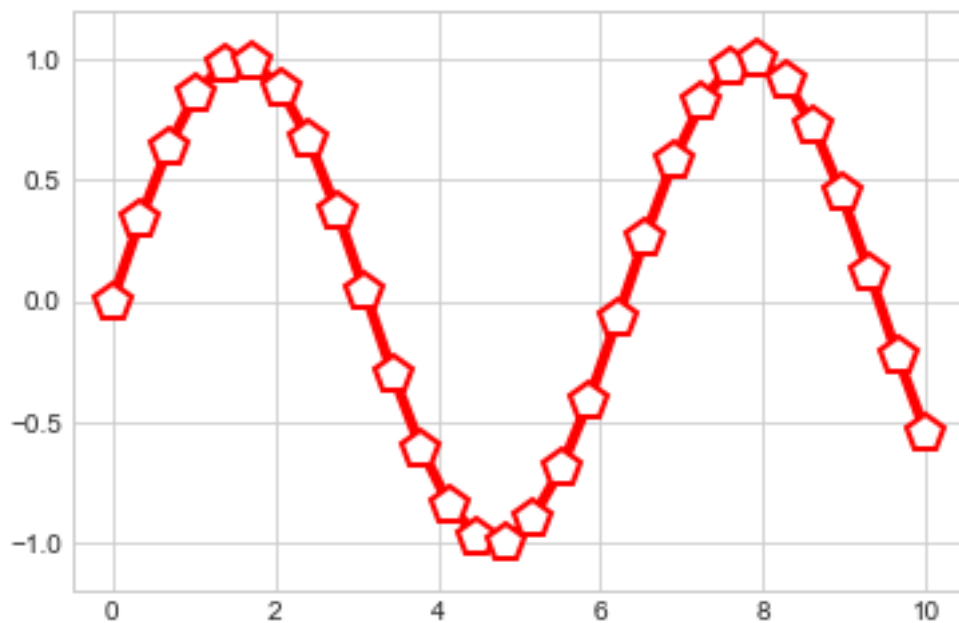
These character codes can be used together with line and color codes to plot points along with a line connecting them

```
[11]: plt.plot(x, y, 'ok')
```

```
[11]: [<matplotlib.lines.Line2D at 0x117567be0>]
```



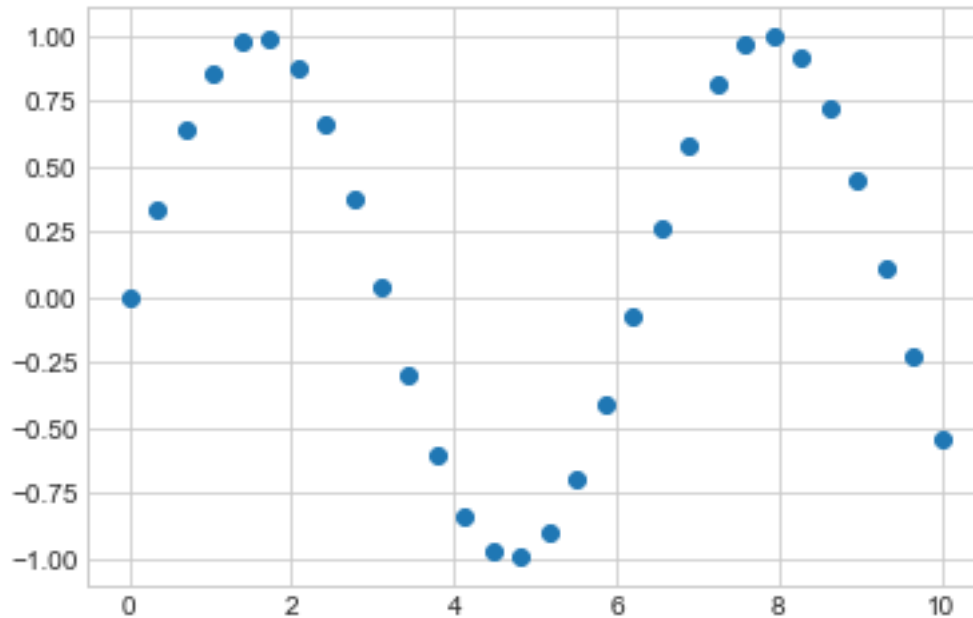
```
[13]: plt.plot(x, y, '-p', color='red',
 markersize=15, linewidth=4,
 markerfacecolor='white',
 markeredgecolor='red',
 markeredgewidth=2)
plt.ylim(-1.2, 1.2);
```



## Scatter Plots with plt.scatter

```
[14]: plt.scatter(x, y)
```

```
[14]: <matplotlib.collections.PathCollection at 0x118835400>
```



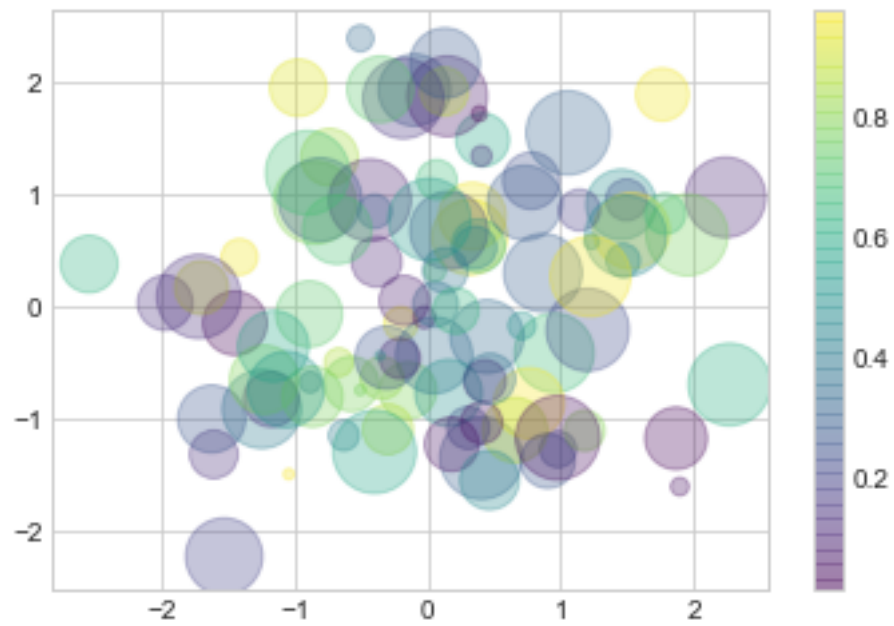
The primary difference of `plt.scatter` from `plt.plot` is that it can be used to create scatter plots. We can control size, color, edge color, etc.

Let's create a random scatter plot with different marks of many colors and sizes. Alpha is an option to make color transparent from 0 to 1.

```
[15]: rng = np.random.RandomState(0)
x = rng.randn(100)
y = rng.randn(100)
colors = rng.rand(100)
sizes = 1000 * rng.rand(100)

plt.scatter(x, y, c=colors, s=sizes, alpha=0.3,
 cmap='viridis')
plt.colorbar() # show color scale
```

```
[15]: <matplotlib.colorbar.Colorbar at 0x11717e2e8>
```



[ ]: